



SESSION 12

NODE.JS IN ACTION (APPLICATION DEVELOPMENT) - II

Learning Objectives

In this session, students will learn to:

- List the steps to upload the Web application to the GitHub repository
- Explain the procedure to deploy the Web application on Render from a GitHub repository

This session carries forward the creation and deployment of the movie Web application from the previous session. It explains additional steps to create pages for the Web application and covers in detail how to deploy the **Wonderland Widescreen** Node.js application using Render.

12.1 Creating More Pages for the Web Application

More pages that are to be created for the Web application are as follows:

- delete-movie-form.html
- update-seats-form.html
- admin-login.ejs
- login.ejs

Let us create these pages and place them in relevant folders for the application to run properly.

➤ delete-movie-form.html

Figure 12.1 shows the design for the delete-movie-form.html page.

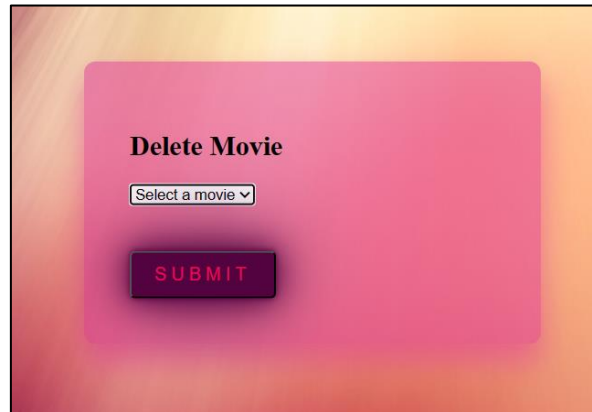


Figure 12.1: delete-movie-form.html

Code Snippet 1 shows the code for delete-movie-form.html.

Code Snippet 1:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Delete Movie</title>
    <style>
      html {
        height: 100%;
      }
      body {
        margin: 0;
        padding: 0;
        background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg");
        background-attachment: fixed;
        background-position: relative;
        background-repeat: no-repeat;
        background-size: cover;
      }

      /* For the center container */
      .center-container {
        position: absolute;
        top: 50%;
        left: 50%;
        transform: translate(-50%, -50%);
      }
    </style>
  </head>
  <body>
    <div class="center-container">
      <div>Delete Movie</div>
      <div>Select a movie</div>
      <div>SUBMIT</div>
    </div>
  </body>
</html>
```

```

/* login box */
.login-box {
    width: 400px;
    padding: 40px;
    background: rgba(226, 53, 157, 0.5);
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(226, 53, 157, 0.5);
    border-radius: 10px;
}

.login-box h1 {
    margin: 0 0 30px;
    padding: 0;
    color: #520340;
    text-align: center;
}

/* User input box */
.login-box .user-box {
    position: relative;
}

.login-box .user-box input {
    width: 100%;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    margin-bottom: 30px;
    border: none;
    border-bottom: 1px solid #fff;
    outline: none;
    background: transparent;
}

.login-box .user-box label {
    position: absolute;
    top: 0;
    left: 0;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    pointer-events: none;
    transition: 0.5s;
}

.login-box .user-box input:focus ~ label,
.login-box .user-box input:valid ~ label {
    top: -20px;
    left: 0;
    color: #520340;
}

```

```

        font-size: 12px;
    }

    /* Submit button */
    .login-box-button {
        position: relative;
        display: inline-block;
        padding: 10px 20px;
        color: #ee0754;
        font-size: 16px;
        text-decoration: none;
        text-transform: uppercase;
        overflow: hidden;
        transition: 0.5s;
        margin-top: 40px;
        letter-spacing: 4px;
        background: #520340;
        border-radius: 5px;
        box-shadow: 0 0 5px #520340, 0 0 25px #520340, 0 0
50px #520340,
        0 0 100px #520340;
    }

    .login-box-button:hover {
        background: #fff;
        color: #520340;
    }
</style>
</head>
<body>
    <div class="center-container">
        <div class="login-box">
            <h2>Delete Movie</h2>

            <form id="movie-form" action="/delete-movie"
method="POST">
                <div class="user-box">
                    <!-- Dropdown for selecting a movie to delete -->
                    <select id="movie-category" name="Movie name">
                        <option value="">Select a movie</option>
                    </select>
                </div>
                <!-- Submit button to delete a movie -->
                <button type="submit" class="login-box-button">
                    <span></span>
                    <span></span>
                    <span></span>
                    <span></span>
                    Submit
                </button>
            </form>

```

```

    </div>
  </div>
  <script>
    async function getMovieList() {
      // To get the movie category dropdown
      const movieCategoryDiv =
document.getElementById("movie-category");
      try {
        // To fetch the movie name from the databse
        const response = await fetch(`/get-all-movies`);
        const data = await response.json();

        // To display the fetched movie name list
        displayMovieList(data);
      } catch (error) {
        // display error
        console.error("Error fetching movies:", error);
      }
    }

    // Function to display the movie list in the dropdown
    function displayMovieList(movies) {
      // To get the movie category dropdown
      const movieCategoryDiv =
document.getElementById("movie-category");

      if (movies.length === 0) {
        // Display a message if no movies are found in the
        // database
        movieCategoryDiv.textContent = `No movies found`;
      } else {
        // Fetch the movie name and display in the dropdown
        movies.forEach((movie) => {
          let option = document.createElement("option");
          option.setAttribute("value", movie["Movie
name"]);
          let optionText =
document.createTextNode(movie["Movie name"]);
          option.appendChild(optionText);

          movieCategoryDiv.appendChild(option);
        });

        movieCategoryDiv.style.display = "block";
        // Show the box when movies are available
      }
    }

    // Call the function to fetch movies when the
    // page loads
    document.addEventListener("DOMContentLoaded",
getMovieList());
  </script>

```

```
</script>
</body>
</html>
```

The code in `delete-movie-form.html`:

- Uses HTML and JavaScript.
- Has `/delete-movie` as the endpoint.
- Uses a drop-down list to allow users select a movie name from the list.
- Defines the `getMovieList` function to fetch all movie names from the MongoDB server using the `/get-all-movies` endpoint.
- Defines the `displayMovieList` function to display the list of movie names in a drop-down list. This function creates `<option>` elements for each movie and appends them to the dropdown.
- Uses the `addEventListener` event listener method to call the `getMovieList` function when the page is loaded.

Save the code with the file name `delete-movie-form.html` in the Templates folder.

➤ **update-seats-form.html**

Figure 12.2 shows the design for the `update-seats-form.html` page.

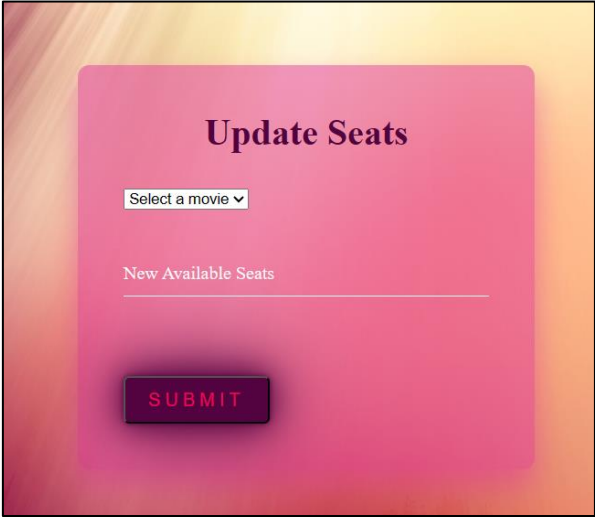


Figure 12.2: update-seats-form.html

Code Snippet 2 shows the code for `update-seats-form.html`. This form updates the movie seats in the database.

Code Snippet 2:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Update Seats</title>

    <style>
      /* Height of the HTML document */
      html {
        height: 100%;
      }

      body {
        margin: 0;
        padding: 0;
        background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg");
        background-attachment: fixed;
        background-position: relative;
        background-repeat: no-repeat;
        background-size: cover;
      }

      /* For the center container */
      .center-container {
        position: absolute;
        top: 50%;
        left: 50%;
        transform: translate(-50%, -50%);
      }

      /* Login box */
      .login-box {
        width: 400px;
        padding: 40px;
        background: rgba(226, 53, 157, 0.5);
        box-sizing: border-box;
        box-shadow: 0 15px 25px rgba(226, 53, 157, 0.5);
        border-radius: 10px;
      }

      /* Heading inside the login box */
      .login-box h1 {
        margin: 0 0 30px;
        padding: 0;
        color: #520340;
        text-align: center;
      }
    </style>
  </head>
  <body>
    <div class="center-container">
      <div class="login-box">
        <h1>Update Seats</h1>
      </div>
    </div>
  </body>
</html>
```

```

    /* User input box */
    .login-box .user-box {
        position: relative;
    }

    /* User input */
    .login-box .user-box input {
    }

    /* Label above the user input */
    .login-box .user-box label {
        /* Label styling */
    }

    /* Button styling */
    .login-box-button {
    }

    /* Hover effect for the button */
    .login-box-button:hover {
    }
</style>
</head>
<body>
    <div class="center-container">
        <div class="login-box">
            <h1>Update Seats</h1>

            <form id="seats-form" action="/update-seats"
method="POST">
                <!-- Dropdown for selecting a movie -->
                <div class="user-box">
                    <select id="movie-category" name="Movie name">
                        <option value="">Select a movie</option>
                    </select>
                </div>
                <br /><br />

                <!-- Input for entering new available seats -->
                <div class="user-box">
                    <input
                        type="number"
                        id="available-seats"
                        name="Available Seats"
                        required=""
                    />
                    <label>New Available Seats</label>
                </div>

                <!-- Submit button -->

```



```

        <button type="submit" class="login-box-button">
            <span></span>
            <span></span>
            <span></span>
            <span></span>
            Submit
        </button>
    </form>
</div>
</div>

<script>
    // Function to fetch movie name from database
    async function getMovieList() {
        const movieCategoryDiv =
document.getElementById("movie-category");
        try {
            const response = await fetch(`/get-all-movies`);
            const data = await response.json();
            displayMovieList(data);
        } catch (error) {
            console.error("Error fetching movies:", error);
        }
    }

    // Function to display the list of movie names
    // in the dropdown
    function displayMovieList(movies) {
        const movieCategoryDiv =
document.getElementById("movie-category");

        if (movies.length === 0) {
            movieCategoryDiv.textContent = `No movies found`;
        } else {
            movies.forEach((movie) => {
                let option = document.createElement("option");
                option.setAttribute("value", movie["Movie
name"]);

                let optionText =
document.createTextNode(movie["Movie name"]);
                option.appendChild(optionText);

                movieCategoryDiv.appendChild(option);
            });

            movieCategoryDiv.style.display = "block";
            // Show the box when movies are available
        }
    }
}

```

```
// Call the function to fetch movies when
// the page loads
document.addEventListener("DOMContentLoaded",
getMovieList());
</script>
</body>
</html>
```

The `update-seats-form.html` form:

- Uses HTML and JavaScript.
- Has `/update-seats` as the endpoint.
- Uses a drop-down list to select a movie.
- Uses an input field to accept available seats.
- Defines the `getMovieList` function to fetch movie names from the database and uses the `/get-all-movies` endpoint. This function uses the `displayMovieList` function to display movie names in a dropdown.
- Defines the `displayMovieList(movies)` function to display the movie names fetched from the database in a drop-down.

Save the code with the file name `update-seats-form.html` in the `Templates` folder.

12.2 Adding Home Page With User Authentication

The application developed so far must be anchored using the home page, `wonderland.html`. This page redirects the user to a specific page depending on whether the user is a guest or an administrator.

➤ `wonderland.html`

Figure 12.3 shows the design of the `Wonderland.html` page.



Figure 12.3: `wonderland.html`

Code Snippet 3 shows the code for `wonderland.html` page.

Code Snippet 3:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Movie List</title>
</head>
<style>
html {
  height: 100%;
}
body {
  text-align: center;
  background:
url("https://cdn.pixabay.com/photo/2017/01/12/17/30/warm-
and-cozy-1975215_1280.jpg");
  background-attachment: fixed;
  background-position: relative;
  background-repeat: no-repeat;
  background-size: cover;
  height: 100%;
  margin: 0;
  place-items: center;
  position: fixed;
  font-size: 20px;
}
.container {
  top: 30%;
  left: 50%;
  width: 50em;
  height: 18em;
  transform: translate(-50%, -50%);
  position: fixed;
  color: rgb(255, 255, 255);
  font-size: 20px;
}
.button {
  background-color: #000;
  border: .5px solid crimson;
  border-radius: 10px;
  color: #fff;
  padding: 8px;
  font-size: 20px;
  cursor:pointer; /* Pointer/hand icon */
  /* display:block; */
}

</style>
```

```
<body>
<div class="container">
  <h1>Welcome to the Wonderland Widescreen</h1>
  <br><br><br>
  <a href="/views/admin-login.ejs"><button
class="button"> Admin Login</button></a>
  <a href="/views/login.ejs"><button class="button">
Guest Login</button></a>

</div>
</body>
</html>
```

This is a simple HTML page that displays two buttons, one each for the administrator and the guest logins. Save the code with the file name `wonderland.html` in the `Templates` folder.

If the user logs in as an administrator, the endpoint will be `admin-login.ejs`. However, if the user logs in as a guest, the endpoint will be `login.ejs`. These files must be created and stored in the `views` folder.

The `.ejs` files are used to provide the HTML markups by embedding the required JavaScript code in the main Node.js application file, `movie-list.js`.

➤ **admin-login.ejs**

Figure 12.4 shows the design of the `admin-login.ejs` page.

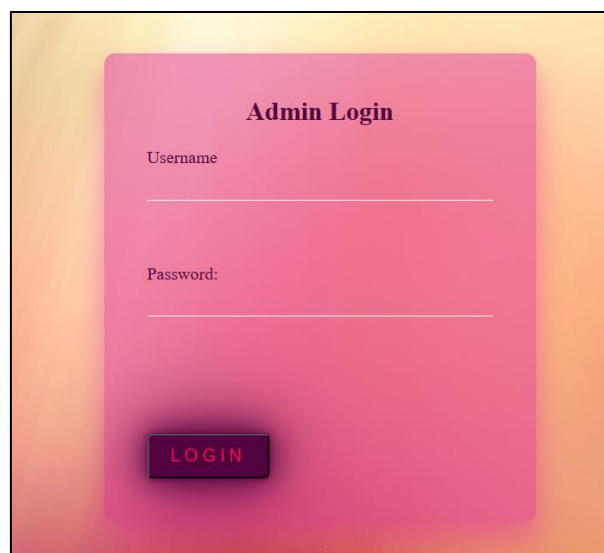


Figure 12.4: admin-login.ejs

Code Snippet 4 shows the code for the `admin-login.ejs` page.

Code Snippet 4:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Admin Login</title>
    <style>
      html {
        height: 100%;
      }

      body {
        margin: 0;
        padding: 0;
        background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg");
        background-attachment: fixed;
        background-position: relative;
        background-repeat: no-repeat;
        background-size: cover;
      }

      .login-box {
        position: absolute;
        top: 50%;
        left: 50%;
        width: 400px;
        padding: 40px;
        transform: translate(-50%, -50%);
        background: rgba(226, 53, 157, 0.5);
        box-sizing: border-box;
        box-shadow: 0 15px 25px rgba(226, 53, 157, 0.5);
        border-radius: 10px;
      }

      .login-box h2 {
        margin: 0 0 30px;
        padding: 0;
        color: #520340;
        text-align: center;
      }

      .user-box {
        position: relative;
        margin-bottom: 20px;
      }
```

```

.user-box input {
    width: 100%;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    margin-bottom: 30px;
    border: none;
    border-bottom: 1px solid #fff;
    outline: none;
    background: transparent;
}

.user-box label {
    position: absolute;
    top: 0;
    left: 0;
    padding: 10px 0;
    font-size: 20px; /* Adjust the font size here */
    color: #fff;
    pointer-events: none;
    transition: 0.5s;
}

.user-box input:focus ~ label,
.user-box input:valid ~ label,
.user-box input:not(:placeholder-shown) ~ label {
    top: -20px;
    font-size: 16px; /* Adjust the font size here */
    color: #520340;
}

.login-box-button {
    position: relative;
    display: inline-block;
    padding: 10px 20px;
    color: #ee0754;
    font-size: 16px;
    text-decoration: none;
    text-transform: uppercase;
    overflow: hidden;
    transition: 0.5s;
    margin-top: 40px;
    letter-spacing: 4px;
    background: #520340;
    border-radius: 5px;
    box-shadow: 0 0 5px #520340, 0 0 25px #520340, 0 0
50px #520340,
        0 0 100px #520340;
}

.login-box-button:hover {

```

```

        background: #fff;
        color: #520340;
    }
</style>
</head>
<body>
    <div class="center-container">
        <div class="login-box">
            <h2>Admin Login</h2>
            <!-- Add a form for admin login -->
            <form id="loginForm" action="/admin-login"
method="POST">
                <div class="user-box">
                    <input type="text" id="username" name="username"
required />
                    <label for="username">Username</label>
                </div>
                <br />
                <div class="user-box">
                    <input type="password" id="password"
name="password" required />
                    <label for="password">Password:</label>
                </div>
                <br />
                <button type="submit" class="login-box-
button">Login</button>
            </form>
        </div>
    </div>
</body>
</html>

```

The admin-login.ejs file:

- Uses /admin-login as the endpoint.
- Uses two input fields one for accepting the username and another for accepting the password.
- Uses a submit button to transfer the execution to the endpoint.

Save the code with the file name admin-login.ejs in the views folder.

➤ login.ejs

Figure 12.5 shows the design of the login.ejs page.

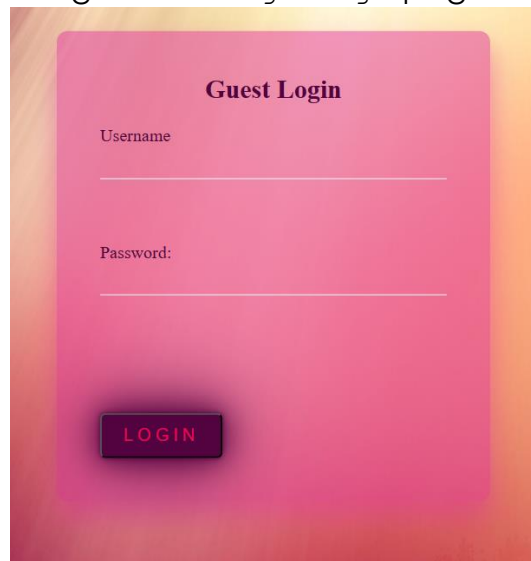


Figure 12.5: login.ejs

Code Snippet 5 shows the code for login.ejs page.

Code Snippet 5:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Guest Login</title>
    <style>
      html {
        height: 100%;
      }

      body {
        margin: 0;
        padding: 0;
        background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg");
        background-attachment: fixed;
        background-position: relative;
        background-repeat: no-repeat;
        background-size: cover;
      }
    </style>
  </head>
  <body>
    <div>
      <h2>Guest Login</h2>
      <input type="text" value="Username" />
      <input type="password" value="Password:" />
      <button type="button" value="LOGIN" />
    </div>
  </body>
</html>
```



```

.login-box {
    position: absolute;
    top: 50%;
    left: 50%;
    width: 400px;
    padding: 40px;
    transform: translate(-50%, -50%);
    background: rgba(226, 53, 157, 0.5);
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(226, 53, 157, 0.5);
    border-radius: 10px;
}

.login-box h2 {
    margin: 0 0 30px;
    padding: 0;
    color: #520340;
    text-align: center;
}

.user-box {
    position: relative;
    margin-bottom: 20px;
}

.user-box input {
    width: 100%;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    margin-bottom: 30px;
    border: none;
    border-bottom: 1px solid #fff;
    outline: none;
    background: transparent;
}

.user-box label {
    position: absolute;
    top: 0;
    left: 0;
    padding: 10px 0;
    font-size: 20px; /* Adjust the font size here */
    color: #fff;
    pointer-events: none;
    transition: 0.5s;
}

.user-box input:focus ~ label,
.user-box input:valid ~ label,
.user-box input:not(:placeholder-shown) ~ label {

```

```

        top: -20px;
        font-size: 16px; /* Adjust the font size here */
        color: #520340;
    }

    .login-box-button {
        position: relative;
        display: inline-block;
        padding: 10px 20px;
        color: #ee0754;
        font-size: 16px;
        text-decoration: none;
        text-transform: uppercase;
        overflow: hidden;
        transition: 0.5s;
        margin-top: 40px;
        letter-spacing: 4px;
        background: #520340;
        border-radius: 5px;
        box-shadow: 0 0 5px #520340, 0 0 25px #520340, 0 0
50px #520340,
        0 0 100px #520340;
    }

    .login-box-button:hover {
        background: #fff;
        color: #520340;
    }
</style>
</head>
<body>
    <div class="center-container">
        <div class="login-box">
            <h2>Guest Login</h2>
            <!-- Add a form for guest login -->
            <form id="loginForm" action="/user-local"
method="POST">
                <div class="user-box">
                    <input type="text" id="username" name="username"
required />
                    <label for="username">Username</label>
                </div>
                <br />
                <div class="user-box">
                    <input type="password" id="password"
name="password" required />
                    <label for="password">Password:</label>
                </div>
                <br />
                <button type="submit" class="login-box-
button">Login</button>

```

```
        </form>
      </div>
    </div>
  </body>
</html>
```

The `login.ejs` file:

- Uses `/user-local` as the endpoint.
- Uses two input fields one for accepting the username and another for accepting the password.
- Uses a submit button to transfer the execution to the endpoint.

Save the code with the file name `login.ejs` in the `views` folder.

12.3 Updating the Main Application File

Finally, update the main Node.js application file, `movie-list.js`. This file acts as the main entry point in the movie application. In this file, the core logic of the application is handled. Routes and middleware are also defined.

➤ `movie-list.js`

Consider the code in Code Snippet 6 for `movie-list.js` file. Add the functionality for user authentication, update, and delete. Note that the code to be added is given in bold.

Code Snippet 6:

```
const express = require("express");
const { MongoClient } = require("mongodb");
const bodyParser = require("body-parser");
const express = require("express");
const { MongoClient } = require("mongodb");
const bodyParser = require("body-parser");
const session = require("express-session");
const passport = require("passport");
const mongoose = require('mongoose');
const LocalStrategy = require("passport-local");
const path = require("path"); // Import the path module
// const uri =
//   "mongodb+srv://prachigupta8585:prachi8585@moviecollectioncluster.4x9rauo.mongodb.net/moviecollectioncluster?retryWrites=true&w=majority"; //MongoDB Atlas Link

//Connect to MongoDB using MongoDB Compass localhost
//const uri = 'mongodb://127.0.0.1:27017/Movie'; // MongoDB connection URI
```

```

const client = new MongoClient(uri);
const app = express();

app.set("view engine", "ejs");

// Use session middleware
app.use(
  session({
    secret: "abc",
    resave: false,
    saveUninitialized: true,
  })
);

app.use(bodyParser.urlencoded({ extended: false }));

// Initialize passport middleware
app.use(passport.initialize());
app.use(passport.session());

// Middleware to check if the user is authenticated
function isAuthenticated(req, res, next) {
  if (req.session && req.session.authenticated) {
    return next();
  } else {
    res.redirect("/login");
  }
}

// Add the admin local strategy
passport.use(
  "admin-local",
  new LocalStrategy(function (username, password, done) {
    if (username === "Admin" && password === "12345") {
      return done(null, { username: "Aptech" });
    }
    return done(null, false, {
      message: "Incorrect admin username or password",
    });
  })
);

passport.serializeUser(function (user, done) {
  done(null, user);
});

passport.deserializeUser(function (user, done) {
  done(null, user);
});

// Add the User name and the password for the guest login

```

```

const users = [
  { id: 1, username: "abc", password: "123" },
  { id: 2, username: "user1", password: "user" },
];

passport.use(
  "user-local",
  new LocalStrategy(function (username, password, done) {
    const user = users.find((u) => u.username === username);
    if (!user) {
      return done(null, false, { message: "Incorrect
username." });
    }
    if (user.password !== password) {
      return done(null, false, { message: "Incorrect
password." });
    }
    return done(null, user);
  })
);

passport.serializeUser(function (user, done) {
  done(null, user.id);
});

passport.deserializeUser(function (id, done) {
  const user = users.find((u) => u.id === id);
  done(null, user);
});

async function main() {
  try {
    await client.connect();
    console.log("Connected to MongoDB");

    const database = client.db();
    const collection =
database.collection("MovieCollection"); // Adjust the
collection name

    // Set up the views folder
    app.set("views", path.join(__dirname, "views"));

    app.get("/", (req, res) => {
      console.log("Received request for /");
      // Serve the HTML page for displaying the login page
for admin and guest
      res.sendFile(__dirname + "/Templates/wonderland.html");
    });

    // Admin get and post

```

```

// Admin login route
app.get("/views/admin-login.ejs", function (req, res) {
  console.log("entered into admin-login page");
  res.render("admin-login");
});

// Admin login form
app.post(
  "/admin-login",
  passport.authenticate("admin-local", {
    // successRedirect: path.join(__dirname, 'Templates',
    'add-movie-form.html'),
    successRedirect: "/admin-dashboard",
    failureRedirect: "/admin-error",
  })
);

// Admin error route
app.get("/admin-error", function (req, res) {
  console.log("getting an admin-error page");
  res.send(
    '<script>alert("Incorrect Admin username or password"); window.location.href = "/";</script>'
  );
});

// Admin dashboard route
app.get("/admin-dashboard", function (req, res) {
  console.log("Entered into admin dashboard page");
  res.sendFile(__dirname + "/Templates/movie-list.html");
});

// User login route
app.get("/views/login.ejs", function (req, res) {
  console.log("Entered into user-login page.");
  res.render("login");
});

// User login form
app.post(
  "/user-local",
  passport.authenticate("user-local", {
    // successRedirect: path.join(__dirname, 'Templates',
    'add-movie-form.html'),
    successRedirect: "/user-dashboard",
    failureRedirect: "/user-error",
  })
);

// User error route
app.get("/user-error", function (req, res) {

```

```

        console.log("Entered into user login error.");
        // res.render("admin-error", { errorMessage: "Incorrect
admin username or password" });
        // Display an alert on the same page
        res.send(
            '<script>alert("Incorrect username or password");
window.location.href = "/";</script>'
        );
    });

    // user dashboard route
    app.get("/user-dashboard", function (req, res) {
        console.log("Entered into user dashboard page");
        res.sendFile(__dirname + "/Templates/book-seats-
form.html");
    });

    // Serve add-movie-form.html
    app.get("/add-movie-form.html", (req, res) => {
        console.log("Received request for /add-movie-
form.html");
        // Serve the HTML form
        res.sendFile(__dirname + "/Templates/add-movie-
form.html");
    });

    // Serve book-seats-form.html
    app.get("/book-seats-form.html", (req, res) => {
        res.sendFile(__dirname + "/Templates/book-seats-
form.html");
    });

    // Serve delete-movie-form.html
    app.get("/delete-movie-form.html", (req, res) => {
        res.sendFile(__dirname + "/Templates/delete-movie-
form.html");
    });

    // Serve update-seats-form.html
    app.get("/update-seats-form.html", (req, res) => {
        res.sendFile(__dirname + "/Templates/update-seats-
form.html");
    });

    app.get("/get-movies", async (req, res) => {
        const category = req.query.category; // Get the
selected category from query parameters

        try {
            // Fetch movies of the selected category from the
database

```

```

        const movies = await collection.find({ Category:
category }).toArray();
        res.status(200).json(movies);
    } catch (error) {
        console.error("Error fetching movies:", error);
        res.status(500).json({ error: "Failed to fetch
movies" });
    }
});

app.get("/get-all-movies", async (req, res) => {

    try {
        // Fetch movies of the selected category from the
database
        const movies = await collection.find().toArray();
        res.status(200).json(movies);
    } catch (error) {
        console.error("Error fetching movies:", error);
        res.status(500).json({ error: "Failed to fetch
movies" });
    }
});

app.get("/get-movie-details", async (req, res) => {
    const movieName = req.query.name; // Get the selected
movie name from query parameters

    try {
        // Fetch movie details based on the selected movie
name from the database
        const movie = await collection.findOne({ "Movie
name": movieName });
        if (movie) {
            res.status(200).json({
                Description: movie["Description"],
                Actors: movie["Actors"],
            });
        } else {
            res.status(404).json({ error: "Movie not found" });
        }
    } catch (error) {
        console.error("Error fetching movie details:",
error);
        res.status(500).json({ error: "Failed to fetch movie
details" });
    }
});

// Handle the form submission and add a movie
app.post("/add-movie", async (req, res) => {

```



```

        try {
            // Insert a new movie document into the MongoDB
collection
            await collection.insertOne(req.body);
            console.log("Added movie to the database");
            // Display an alert on the same page and redirect to
admin dashboard
            res.send('<script>alert("Movie added successfully");
window.location.href = "/admin-dashboard";</script>');
        } catch (error) {
            console.error("Error adding movie:", error);
            // Render an error message on the same page
            res.status(500).send("<h2>Failed to add the
movie</h2>");
        }
    });

// Handle booking seats
app.post("/book-seats", async (req, res) => {
    try {
        // This checks if the user is an admin or a guest based
on the authentication
        const isAdmin = req.isAuthenticated() &&
req.user.username === "Aptech";

        // Retrieve movie information from the request
        const movieNameToBook = req.body["Movie name"];
        const seatsToBook = parseInt(req.body["seats-to-book"]);

        // Check if the movie exists
        const existingMovie = await collection.findOne({ "Movie
name": movieNameToBook });

        if (!existingMovie) {
            console.log("Movie not found in the database.");
            return res.send("Movie not found in the database.");
        }

        // Retrieve the available seats from the existing movie
document
        const availableSeats = existingMovie["Available Seats"];
//Check field name

        if (seatsToBook <= availableSeats) {
            // Calculate the updated available seats
            const updatedAvailableSeats = availableSeats -
seatsToBook;

            // Update the document with new available seats
            const result = await collection.updateOne(
                { "Movie name": movieNameToBook },

```

```

        { $set: { "Available Seats": updatedAvailableSeats }
    } //Check field name
    );

    // This will redirect to the route based on the users
    role admin or user using the conditional (ternary) operator
    concept
    const redirectRoute = isAdmin ? '/admin-dashboard' :
    '/user-dashboard';

    if (result.modifiedCount === 1) {
        // Display an alert and redirect to the appropriate
route
        const alertMessage = `Booking successful for
    ${seatsToBook} seat(s) in ${movieNameToBook}`;
        return res.send(`
            <script>
                alert("${alertMessage}");
                window.location.href = "${redirectRoute}";
            </script>
        `);
    } else {
        console.log("Failed to update available seats");
        const errorMessage = "Failed to update available
seats";
        // Display an alert on the same page
        return res.send(`
            <script>
                alert("${errorMessage}");
                window.location.href = "${redirectRoute}";
            </script>
        `);
    }
    } else {
        console.log(`Not enough seats available for
    ${seatsToBook} seat(s) in ${movieNameToBook}`);
        return res.send(`Not enough seats available for
    ${seatsToBook} seat(s) in ${movieNameToBook}`);
    }
    } catch (error) {
        console.error("Error booking seats:", error);
        return res.status(500).send("Failed to book seats");
    }
    });

    // Handle delete movie
    app.post("/delete-movie", async (req, res) => {
        const movieNameToDelete = req.body["Movie name"];

        try {
            // Check if the movie exists

```

```

const existingMovie = await collection.findOne({
  "Movie name": movieNameToDelete,
});

if (!existingMovie) {
  console.log("Movie not found in the database");
  res.send("Movie not found in the database");
} else {
  // Delete the movie from the collection
  const result = await collection.deleteOne({
    "Movie name": movieNameToDelete,
  });

  if (result.deletedCount === 1) {
    console.log("Movie deleted successfully");
    // Display an alert on the same page and redirect
    to admin dashboard
    res.send(
      '<script>alert("Movie deleted successfully");
window.location.href = "/admin-dashboard"</script>');
  } else {
    console.log("Failed to delete the movie");
    // Display an alert on the same page
    res.send(
      '<script>alert("Failed to delete the movie");
window.location.href = "/"</script>'
    );
  }
} catch (error) {
  console.error("Error deleting the movie:", error);
  res.status(500).send("Failed to delete the movie");
}
});

// Handle updating available seats
app.post("/update-seats", async (req, res) => {
  const movieNameToUpdate = req.body["Movie name"];
  const newAvailableSeats = parseInt(req.body["Available
Seats"]); // Make sure the field name matches your MongoDB
collection

  try {
    // Check if the movie exists
    const existingMovie = await collection.findOne({
      "Movie name": movieNameToUpdate,
    });

    if (!existingMovie) {
      console.log("Movie not found in the database");
      // Display an alert on the same page

```

```

        res.send(
            '<script>alert("Movie not found in the
database"); window.location.href = "/";</script>'
        );
    } else {
        // Update the available seats for the movie
        console.log("Existing movie:", existingMovie);

        const result = await collection.updateOne(
            { _id: existingMovie._id }, // Use _id to
uniquely identify the movie
            { $set: { "Available Seats": newAvailableSeats }
        } // Make sure the field name matches your MongoDB collection
        );

        console.log("Update operation result:", result);

        if (result.modifiedCount === 1) {
            const alertMessage = `Updated available seats for
${movieNameToUpdate} successfully`;
            console.log(alertMessage);
            // Display an alert on the same page and redirect
to admin dashboard
            res.send(
                '<script>alert("${alertMessage}")';
window.location.href = "/admin-dashboard";</script>');
            } else {
                console.log("Failed to update available seats");
                res.status(500).send("Failed to update available
seats");
            }
        }
    } catch (error) {
        console.error("Error updating available seats:",
error);
        res.status(500).send("Failed to update available
seats");
    }
    });

    //logout
    app.get("/logout", function (req, res) {
        console.log("Logout page activated.");
        res.sendFile(__dirname + "/Templates/wonderland.html");
    });
} catch (error) {
    console.error("Error connecting to MongoDB:", error);
}
}

main().catch(console.error);

```

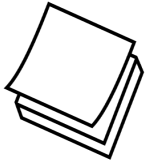
```
app.listen(process.env.PORT || 3000, process.env.IP ||
"0.0.0.0", () => {
  console.log("Server is running");
});
```

The `movie-list.js` file:

- Uses `app.use` to set up session middleware using the `express-session` package. Here,
 - `secret` is a key used to sign the session ID cookie.
 - `resave` and `saveUninitialized` are session options.
 - Initializes and uses `passport` middleware for handling user authentication.
 - Defines the `isAuthenticated` function to check for user authentication and redirect the user to the login page if the user is not authenticated.
 - Uses the `admin-local` and `user-local` passport strategies to authenticate the credentials.
 - Uses the `serialize` and `deserialize` functions to define how user objects are serialized into the session and deserialized from the session.
 - Defines additional routes for handling HTTP requests. The routes are:
 - `/delete-movie`: This route is used for handling delete movie.
 - `/update-seats`: This route is used for handling updating available seats.
 - `/logout`: This route is used to logout from the application.
- Admin login routes:
- `/views/admin-login.ejs`: This route is used to redirect to the admin login page.
 - `/admin-login`: This route is used to handle admin login form submission.
 - `/admin-error`: This route is used to handle login errors such as incorrect credentials.
 - `/admin-dashboard`: This route is used to redirect to the admin dashboard.
- User login routes:
- `/views/login.ejs`: This route is used to redirect to the login page of guest.
 - `/user-local`: This route is used to handle guest form submission.
 - `/user-error`: This route is used to handle login errors such as incorrect credentials.

- /user-dashboard: This route is used to redirect to the guest dashboard.
- Manages the logout process by accessing the /logout route, which clears the session and redirects them to the home page.

Save the code. This completes the coding for the application.



To add the **Logout** button, in the `movie-list.html` page add this code before the ending of `div` tag:

```
<a href="/logout"><button class="button">Logout</button>
</a>
```

12.4 Uploading the Application to the GitHub Repository

To be accessible globally, the application must be deployed on the Internet. The GitHub repository serves as a global storage that hosts the application files. This repository can store codes, files, and version histories of applications stored on it. To upload the movie application to the GitHub repository, create an exclusive repository space for the application in GitHub.

Refer to Appendix B for a detailed description of how to:

- Create a repository in GitHub.
- Upload the application resources to the repository.

12.5 Deploying the Application on Render from a GitHub Repository

Render is a cloud computing platform that helps developers to deploy their applications on the cloud. Refer to session 10 to deploy the application using Render. Deploy the movie application uploaded to the GitHub repository using Render. When the deployment is complete, copy the URL from the Dashboard. The URL, <https://movie-application-o2ed.onrender.com> is copied to the clipboard.

12.6 Accessing the Application in Web Browser

The Wonderland Widescreen application is now deployed and is ready to be accessed in the Web browser using the URL.

Perform the given steps:

1. Open the Web browser and paste the copied URL. The home page of the movie application is displayed as shown in Figure 12.6.



Figure 12.6: Home Page of Movie Application

Guest Login

2. Click **Guest Login**.
3. The guest login form is displayed as shown in Figure 12.7.

The image shows a guest login form. The form is a light pink rectangle with rounded corners, centered on a background with a warm orange-to-yellow gradient. At the top of the form, the text "Guest Login" is written in a dark pink serif font. Below this, there are two input fields. The first is labeled "Username" in a small, dark pink sans-serif font, and it contains the text "User1". The second is labeled "Password:" in the same font, and it contains the text "user". To the right of the password field is a small icon of an eye with a diagonal line through it, indicating a toggle for password visibility. At the bottom of the form, there is a dark pink button with the word "LOGIN" in white, uppercase, sans-serif font.

Figure 12.7: Guest Login Page

4. To provide the username, in the **Username** box, type **user1**.
5. To provide the password, in the **Password** box, type **user**, and click **Login**. The **Book Your Seats** page appears.
6. To book a movie, click **Action**, **Comedy**, or **Drama** button. The movie list appears as shown in Figure 12.8. Select a movie to see the description of the movie.

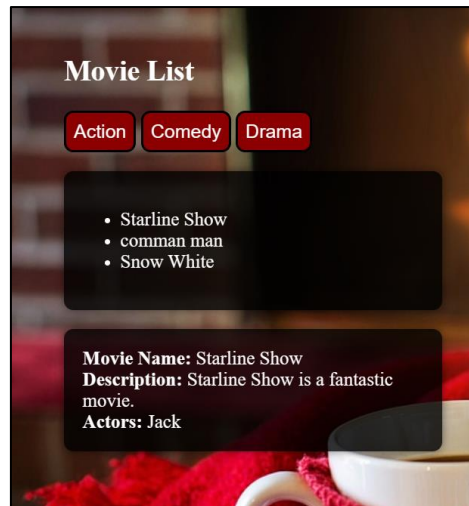


Figure 12.8: Book Your Seats - Movie List Section

7. Then, select a movie from the drop-down and enter the number of seats as shown in Figure 12.9.

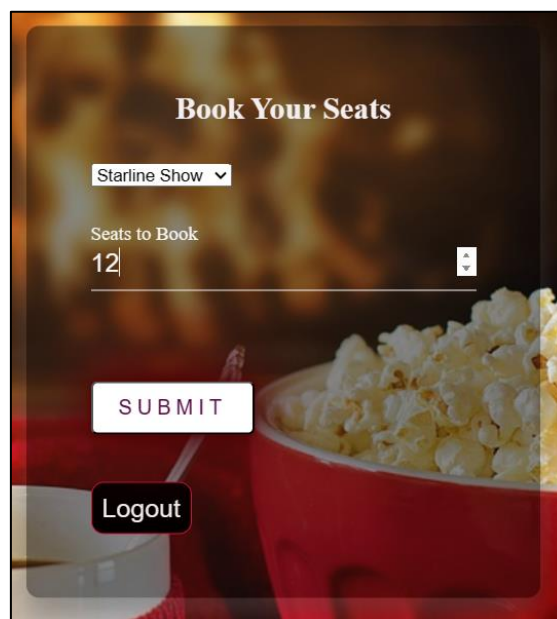


Figure 12.9: Book Your Seats

8. Click **SUBMIT**.

Figure 12.10 shows the message that will appear after the seats are booked and the data is updated to the database.

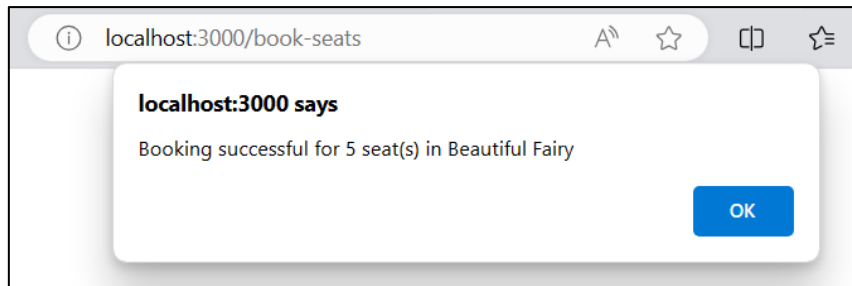


Figure 12.10: Booking Seats Confirmation

Admin Login

9. Click **Admin Login** as shown in earlier Figure 12.6.
The admin login form is displayed as shown in Figure 12.11.
10. To provide the username, in the **Username** box, type **Admin**.
11. To provide the password, in the **Password** box, type **12345**, and click **Login**.

A screenshot of the Admin Login form. The form has a pink background with a white border. It contains two input fields: 'Username' with the value 'Admin' and 'Password' with the value '12345'. There is a 'LOGIN' button at the bottom. A small eye icon is visible next to the password field.

Figure 12.11: Admin Login Form

Admin login is done successfully and the dashboard appears as shown in Figure 12.12.

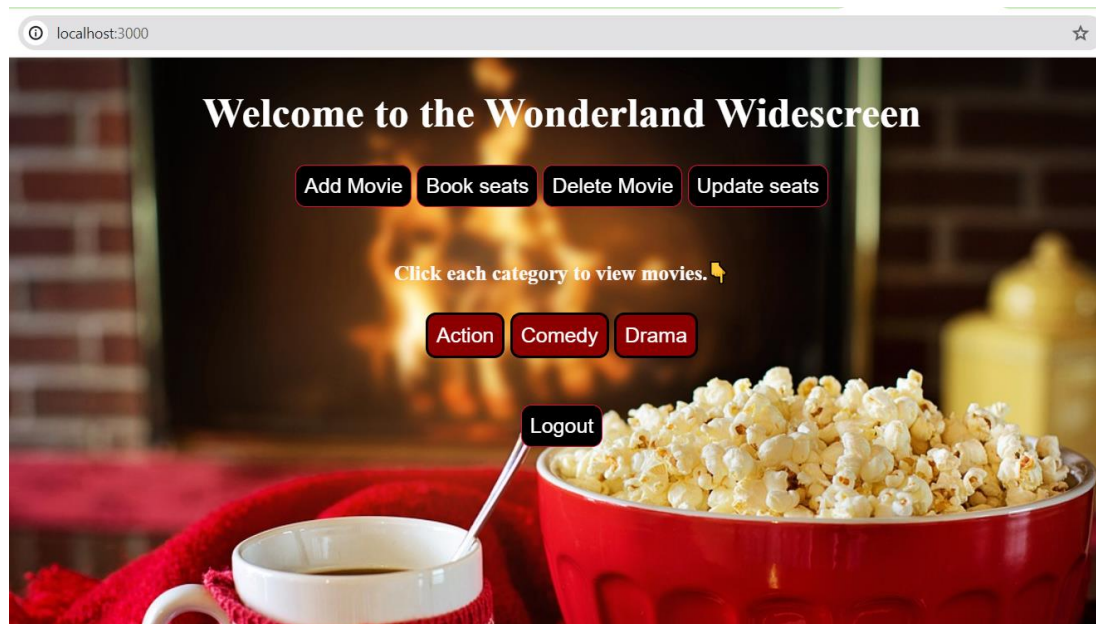


Figure 12.12: Admin Login Dashboard

Add Movie

12. Now, to add a new movie click **Add Movie**. The **Add movie Details** form is displayed as shown in Figure 12.13.

A form titled 'Add Movie Details' with a pink background. It contains several input fields with pre-filled text: 'Movie Name' (Beautiful Fairy), 'Screen No' (2), 'Show Time' (11.00), 'Available Seats' (120), and 'Actors Name' (Jack and Rose). There is a 'Movie Description' field with the text 'Beautiful Fairy is a Drama movie.' and a dropdown menu set to 'Drama'. A 'SUBMIT' button is at the bottom.

Figure 12.13: Add Movie Details Page

13. Enter the details in the **Add Movie Details** page as shown in Figure 12.13.
14. Click **SUBMIT**.

Figure 12.14 shows the message that will appear after the movie is added to the database.

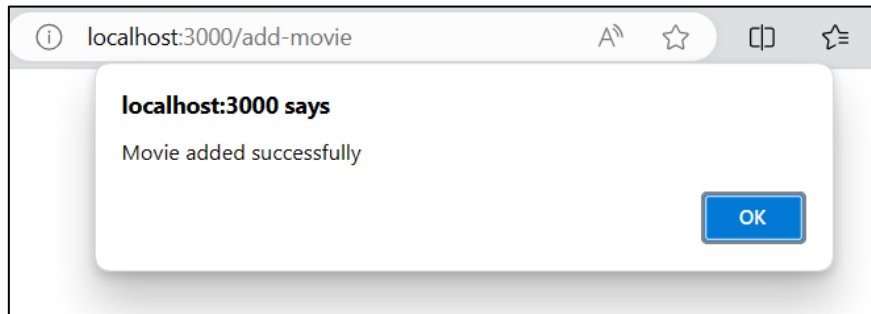


Figure 12.14: Add Movie Confirmation

Book Seats

15. To book a movie click **Book seats** as shown in earlier Figure 12.12.



Booking the seats functionality is the same for both guest and admin login.

Update Seats

16. To update movie seat, click **Update seats** as shown in earlier Figure 12.12.

Update Seats form is displayed as shown in Figure 12.15.

A screenshot of a web form titled 'Update Seats'. The form has a pink background and is set against a gradient orange-to-yellow background. At the top, the title 'Update Seats' is in a bold, dark purple font. Below the title is a dropdown menu with 'Beautiful Fairy' selected. Underneath is a label 'New Available Seats' followed by a text input field containing the number '50'. To the right of the input field is a small square button with a double-headed vertical arrow. At the bottom of the form is a large, dark purple button with the word 'SUBMIT' in white capital letters.

Figure 12.15: Update Seats Page

17. Enter the details in the **Update Seats** page as shown in Figure 12.15 and then click **SUBMIT**.

Figure 12.16 shows the message that will appear after the movie is updated in the database.

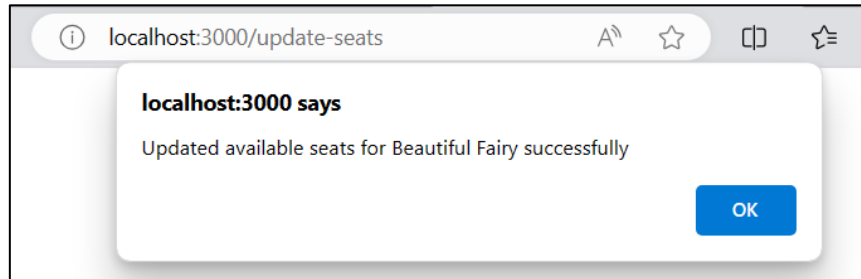


Figure 12.16: Update Movie Confirmation

Delete Movies

18. To delete a movie, click **Delete Movie** as shown in earlier Figure 12.12. **Delete Movie** form is displayed as shown in Figure 12.17.

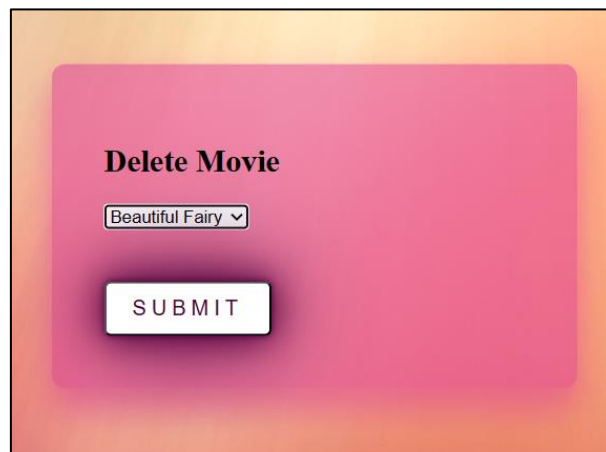


Figure 12.17: Delete Movie Page

19. Enter the details in the **Delete Movies** page as shown in Figure 12.17 and click **SUBMIT**.

Figure 12.18 shows the message that will appear after the movie is deleted from the database.

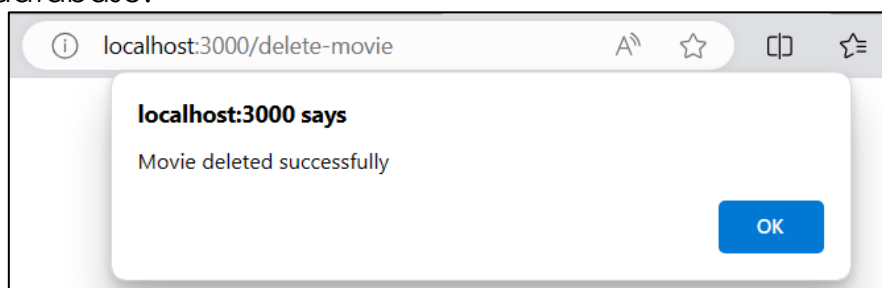


Figure 12.18: Delete Movie Confirmation

12.7 Summary

- Embedded JavaScript files are used to provide the HTML markups. The scripts required are embedded in the main Node.js application file.
- The `.ejs` files are stored in the `views` folder.
- Middleware for maintaining user session and authentication is added to the main Node.js application file using the `passport` middleware.
- `app.use` from the `express-session` package is used to set up session middleware.
- A Web application can be uploaded to the GitHub repository for better collaboration and version control.
- Render can be used to deploy Web applications on the cloud.

Test Your Knowledge

1. Which of the following accounts are required to deploy an application on Render?

- a) Gmail
- b) GitHub
- c) Node.js
- d) Express

2. Consider the given code snippet.

```
app.get('/', (req , res) => {  
  const data = {  
    Cricketer_Name: 'Chris Gayle',  
    Cricketer_Hobby: 'Cricket',  
  };  
  res.render('index', data) ;  
});
```

Which of the following code snippets will you use to display Cricketer_Name and Cricketer_Hobby in a .html file?

- a) `<body>`
 `<h1> <%= Cricketer_Name %> </h1>`
 `<h2> <%= Cricketer_Hobby %> </h3>`
 `</body>`
- b) `<body>`
 `<h1> <% Cricketer_Name %> </h1>`
 `<h2> <% Cricketer_Hobby %> </h3>`
 `</body>`
- c) `<body>`
 `<h1> </= Cricketer_Name /> </h1>`
 `<h2> </= Cricketer_Hobby /> </h3>`
 `</body>`
- d) `<body>`
 `<h1> <%= &Cricketer_Name %> </h1>`
 `<h2> <%= &Cricketer_Hobby %> </h3>`
 `</body>`

3. Which option will you choose on the dashboard to deploy a Web application on Render?

- a) Static Sites
- b) Private Services
- c) New Web Service
- d) Blueprints

4. Which command will you enter in the start command section while deploying a Web application on Render?

- a) `npm file_name.js`
- b) `npm install`
- c) `npm file_name.ejs`
- d) `npm file_name.html`

5. What does `res.status(500)` represent in Node.js?

- a) Completed successfully
- b) Not found
- c) Exceptional
- d) Internal server error

Answers to Test Your Knowledge

1	a, b
2	a
3	c
4	a
5	d

Try It Yourself

1. Refer to the **Student Management System** application developed in Session 11.
 - a) Create an HTML form to delete a student selected from a list of students. Add a listbox to list all the student names. Allow the user to select a student from the list. Add a **Delete** button to delete the selected student.
 - b) On the home page, add a button **Delete**. When this button is clicked the application must redirect the user to the delete form.
 - c) Implement user authentication using admin and guest login.
 - a. Create an admin login page. The user should be redirected to the admin dashboard.
 - b. Include the add, update, and delete functionality for the admin dashboard.
 - c. Create a student login page. The user should be redirected to the student dashboard.
 - d. Include only the update functionality for the student login from which the students can update marks.
 - d) Create a Github repository for the application.
 - e) Upload the application to the Github repository.
 - f) Deploy the application using Render.
 - g) Execute the application from a browser.

Appendix A

RETRIEVE CONNECTION STRING FROM MONGODB ATLAS

MongoDB Atlas is a cloud-based service that requires an account to access its servers. With an Atlas account, the Atlas dashboard can be accessed to deploy MongoDB clusters, create database users, and manage databases.

Create an Atlas Account

To create an Atlas account, use any one of the three options:



Google account is the preferred method. However, email address can also be used.

To create the Atlas account:

1. Open the browser and navigate to the URL:
<https://www.mongodb.com/cloud/atlas/register>

The **MongoDB Atlas Landing** page opens as shown in Figure 1. To sign up using the preferred method or Google account, scroll down and click **Sign up with Google**.

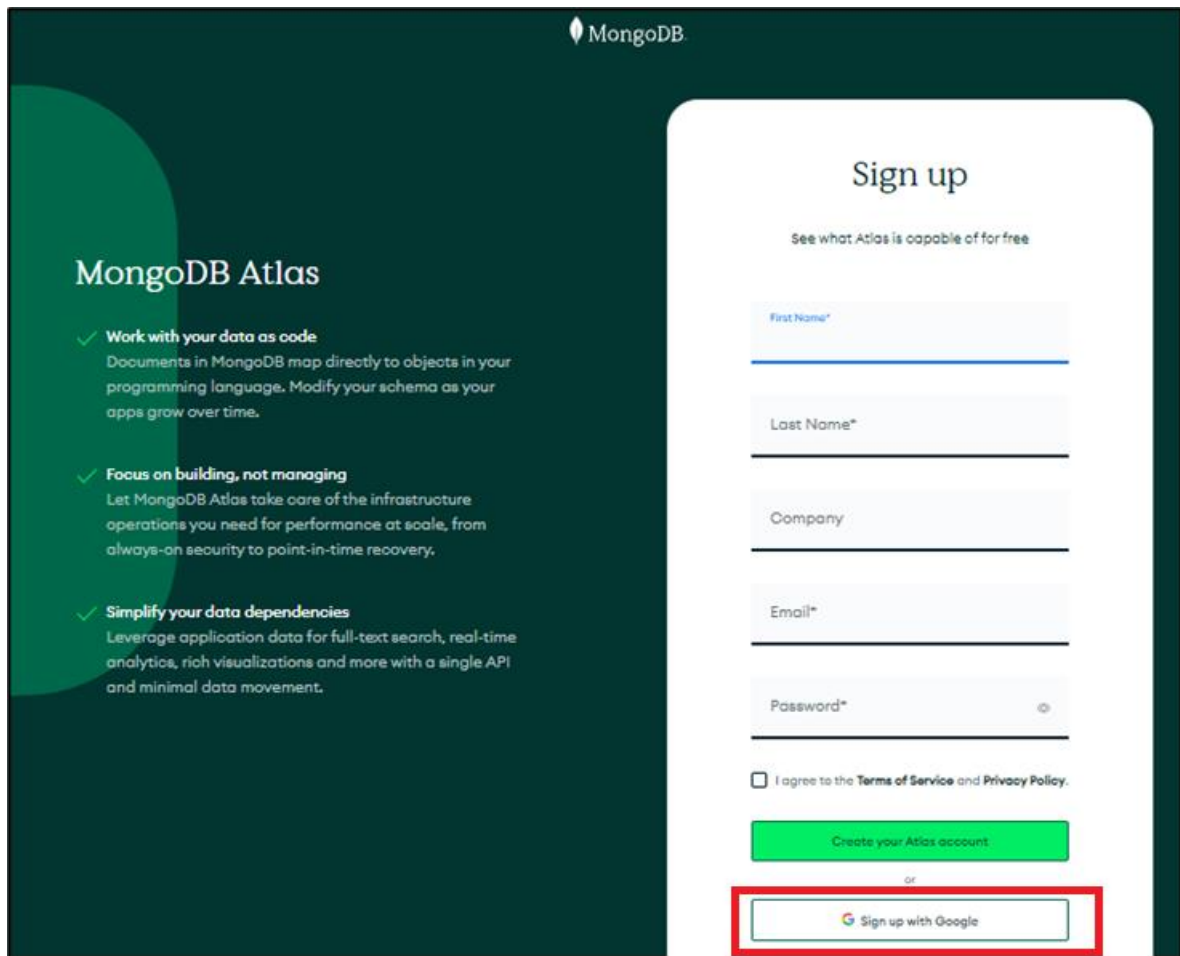


Figure 1: MongoDB Atlas Landing Page

2. The **Accept Privacy Policy and Terms of Service** page opens as shown in Figure 2. To understand how the data is collected, used, and shared, review the privacy policy and terms of service.

To proceed, select the **I accept the Privacy Policy and Terms of Service** check box.

Click **Submit**.

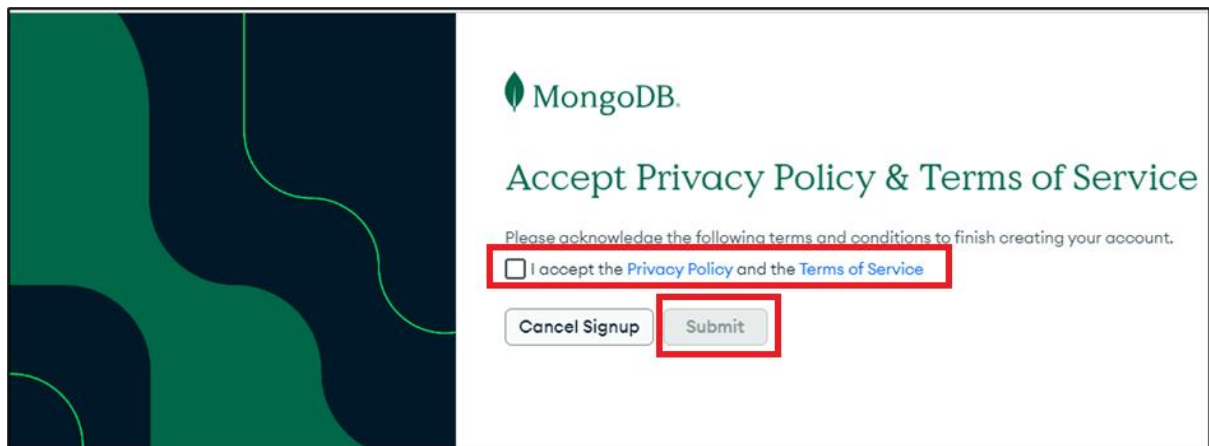


Figure 2: Accept Privacy Policy and Terms of Service Page

3. This completes the sign-up process and welcomes the user to the account as shown in Figure 3.

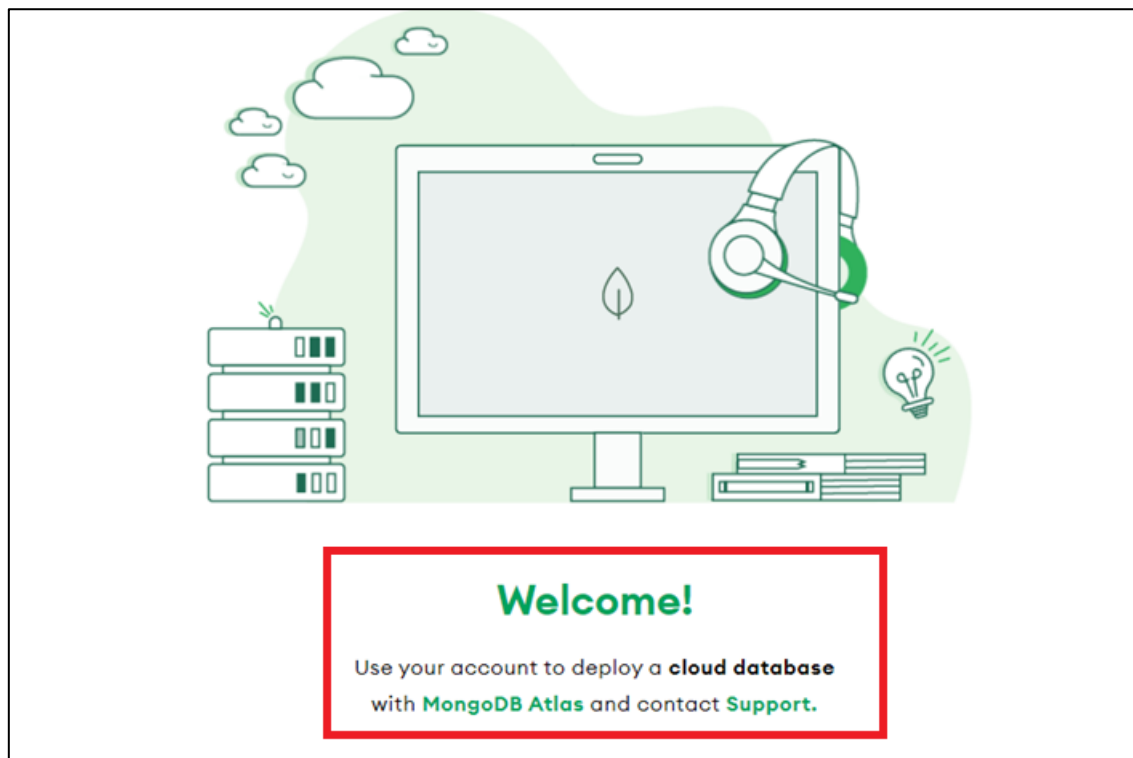


Figure 3: Welcome Page

4. Atlas automatically creates a default organization and a project. For first-time users, Atlas prompts them to respond to a few queries related to the user and the project as shown in Figure 4.

Respond to all the queries appropriately and click **Finish**. This completes the Atlas account creation process.

Welcome to Atlas! 🌱

Tell us a few things about yourself and your project.

What is your goal today?
Your answer will help us guide you to successfully getting started with MongoDB Atlas.

- ☐ Build a new application
- ☐ Migrate an existing application
- ☐ Explore what I can build
- ☒ Learn MongoDB

What type of application are you building?

I'm just exploring ▼

What is your preferred language?
We'll use this to customize code samples and content we share with you. You can always change this later.

Other ▼

Finish

Figure 4: Welcome to Atlas Page

Deploy a Free Cluster

After creating an Atlas account, the next step is to create a cluster. A cluster is a group of servers that work together to store and manage data. To create a MongoDB cluster in Atlas, the user must specify the required size and configuration, and Atlas takes care of the rest.

M0 clusters are a free, entry-level option for MongoDB users who are learning the database or developing small, proof-of-concept applications. These clusters are limited in terms of storage and features, but they are a great way to get started with MongoDB without any upfront costs. These clusters are small-scale, but they never expire and they provide access to a subset of Atlas features.

To deploy M0 cluster:

1. From the **Deploy your database** page shown in Figure 5, select the **M0** option.

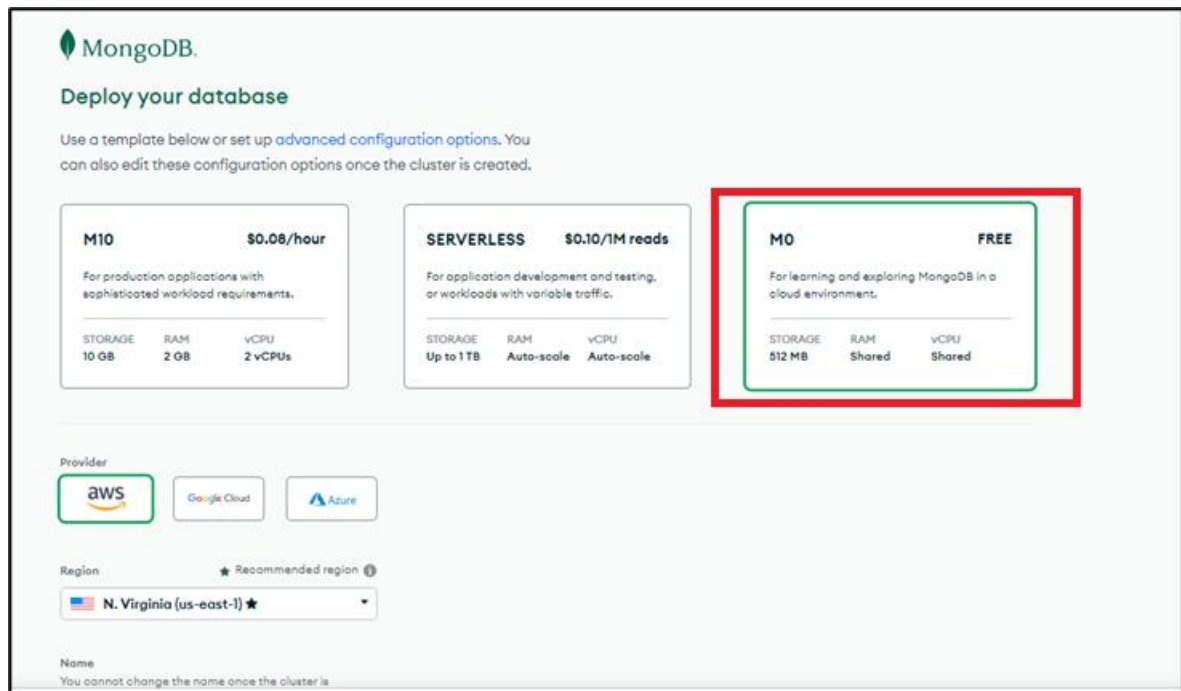


Figure 5: Deploy your database Page

2. Scroll down to make more selections as shown in Figure 6.
 - Select the preferred **Provider**.
Atlas supports M0 free clusters on Amazon Web Services (AWS), Google Cloud Platform (GCP), and Microsoft Azure. In this case, the **AWS** option is selected.
 - Select the preferred **Region**.
Atlas displays only the cloud provider regions that support M0 free clusters. Here, **N. Virginia (us-east-1)** is selected.
 - Enter a name for the cluster in the **Name** box. Users can specify any name for the cluster, as long as it contains American Standard Code for Information Interchange or ASCII letters, numbers, and hyphens. Retain **Cluster0** as given here.
 - Click **Create**.

The screenshot shows a 'Create Cluster' form with the following fields and options:

- Provider:** A selection area with three buttons: 'aws' (highlighted with a red box), 'Google Cloud', and 'Azure'.
- Region:** A dropdown menu showing 'N. Virginia (us-east-1)' with a star icon indicating it is the recommended region. The entire dropdown is highlighted with a red box.
- Name:** A text input field containing 'Cluster0'. A note above the field states: 'You cannot change the name once the cluster is created.' The input field is highlighted with a red box.
- Tag (optional):** A section for adding tags, with a note: 'Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)' Below this are two input fields: 'Select or enter key' and 'Select or enter value'.
- FREE:** A green 'Create' button at the bottom, highlighted with a red box.

Figure 6: Create Cluster

3. The **Security Quickstart** page opens as shown in Figure 7 with the success message.

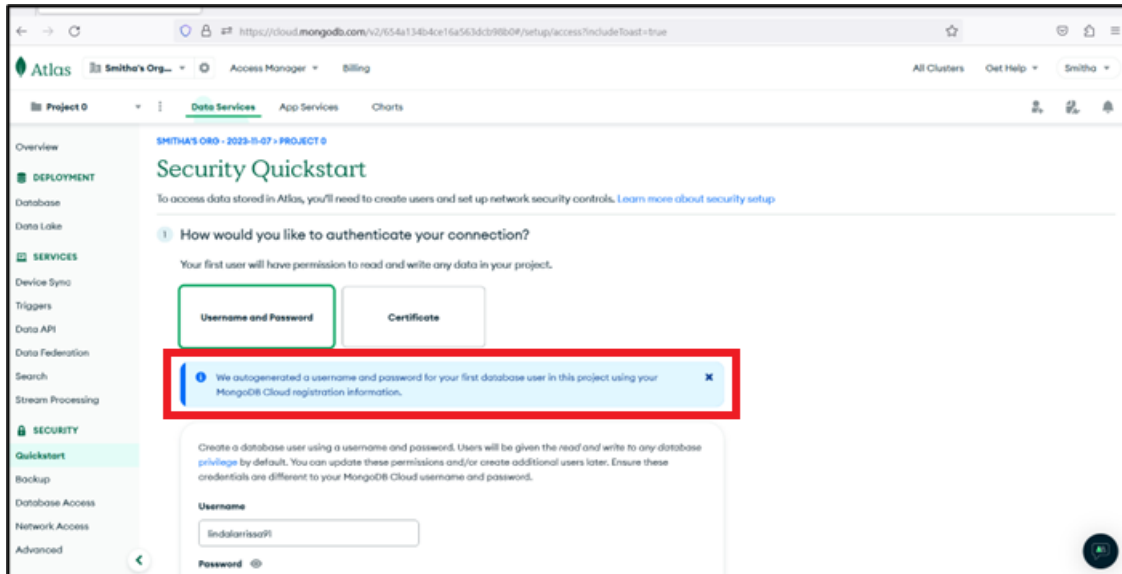


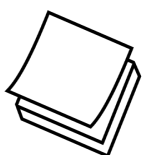
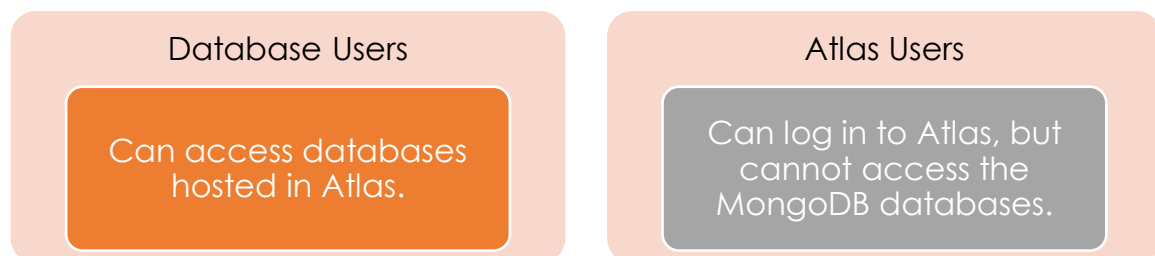
Figure 7: Success Message on the Security Quickstart Page

This completes the deployment of free cluster M0.

Create a Database User for the M0 Cluster

To protect data, Atlas requires users to authenticate as valid MongoDB database users before they can access the cluster. To allow users to access the MongoDB database on the cluster, a database user must be created after deploying the cluster. The database user can then be granted permission to access the cluster and the database hosted on the cluster. The database user accounts can be created for different users and roles, and each account can be granted different permissions.

There is a difference between database users and Atlas users in MongoDB Atlas:



A database user account can ONLY be created by a user who has either Organization Owner or Project Owner permission.

MongoDB Atlas autogenerates a username and password for the first database user in the project using the already submitted MongoDB Cloud registration information.

To manually add a user account:

1. On the **Security Quickstart** page, as shown in Figure 8, enter the new username and credentials.
 - For this session, enter the username as **lindalarrissa91** and password as **linda91**. This username and password combination is used to grant a user the access to databases and collections in the cluster within the Atlas project.
 - Click **Create User**. This creates a database user.

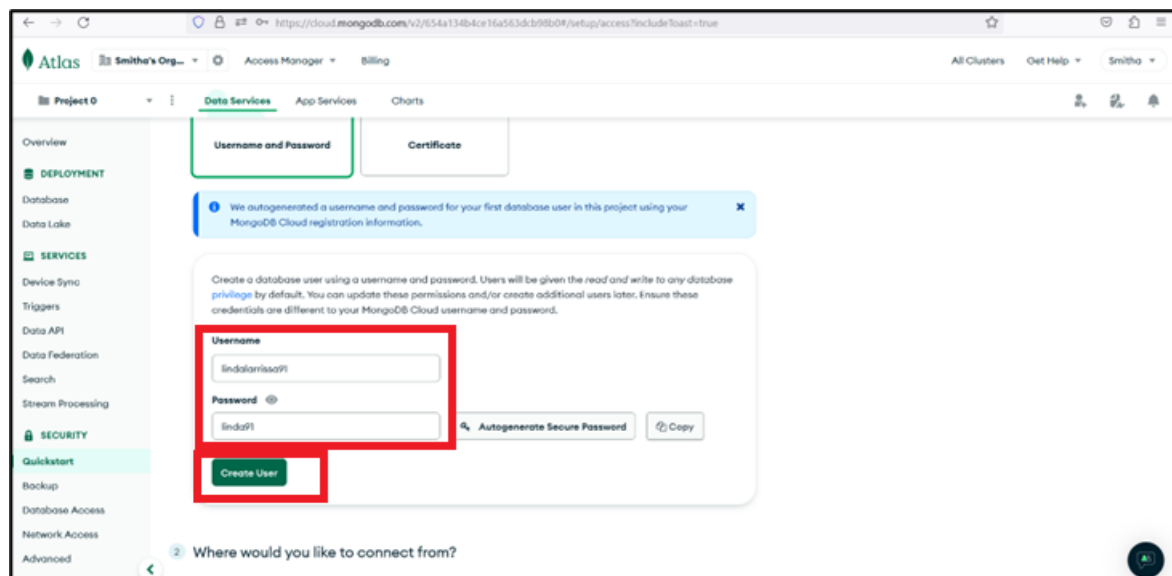


Figure 8: Create User

Add a Trusted IP Address to the Authorized IP Access List

After creating a database user, the deployed cluster is ready to be connected. Each device that connects to a network is assigned a unique identifier called an Internet Protocol or IP address. It is used to identify and locate devices on the network, and to route data between devices.

In MongoDB Atlas, an IP access list is a list of trusted IP addresses that are allowed to connect to a MongoDB Atlas cluster. To secure the MongoDB Atlas cluster, access should be restricted only to specified IP addresses. By limiting access to specific IPs, unauthorized connections can be prevented so as to ensure that only trusted sources can connect to the cluster.

To add an IP Address:

1. For this session, under **Add entries to your IP Access List**, in the **IP Address** box, enter **0.0.0.0/0**, and click **Add Entry** as shown in Figure 9.

The specified IP Address can be retrieved after deploying the application on the Render platform. Therefore, a temporary IP address is entered that allows connections from any network.

Click **Finish and Close**.

Add entries to your IP Access List

Only an IP address you add to your Access List will be able to connect to your project's clusters. You can manage existing IP entries via the [Network Access Page](#).

IP Address Description

Enter IP Address Enter description Add My Current IP Address

Add Entry

IP Access List	Description
0.0.0.0/0	

EDIT REMOVE

Finish and Close

Figure 9: Add entries to your IP Access List Page

2. The **Congratulations on setting up access rules!** message box appears. Click **Go to Overview** as shown in Figure 10.

Congratulations on setting up access rules!

You will now be able to connect to your deployments. You can continue to add and update access rules in [Database Access](#) and [Network Access](#).

☒ Hide Quickstart guide in the navigation. You can visit [Project Settings](#) to access it in the future.

Go to Overview

Figure 10: Congratulations on setting up access rules Message

Get the Connection String

To retrieve the connection string of the database deployment:

1. Click **Database** in the upper-left corner of the Atlas screen.
The **Database Deployments** page opens as shown in Figure 11. To deploy the desired deployment, click **Connect** for **Cluster0**.

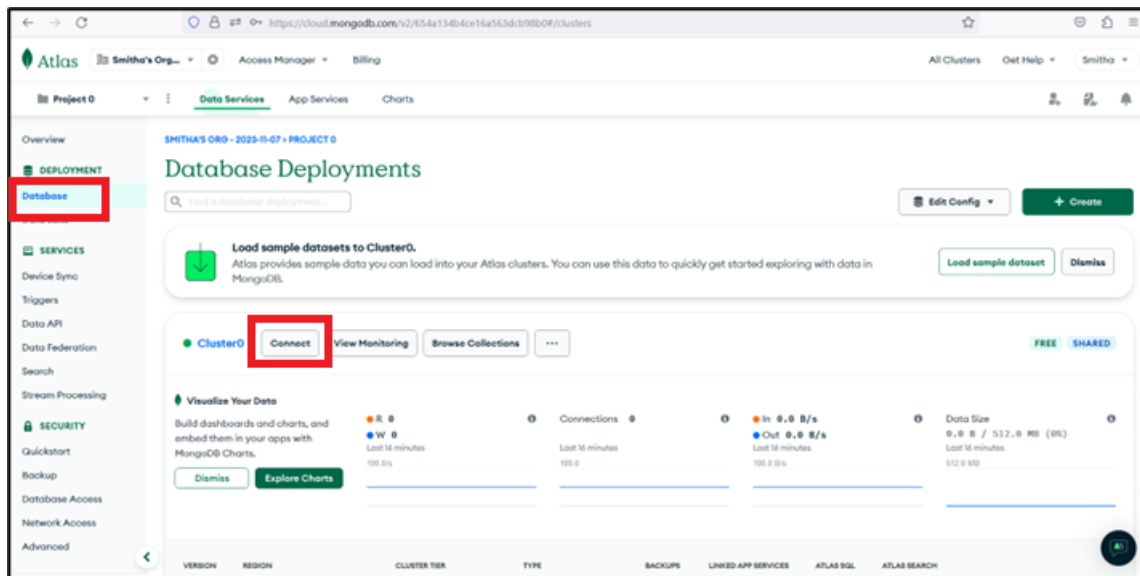


Figure 11: Database Deployments Page

2. The **Choose a connection method** page opens as shown in Figure 12. To connect using the MongoDB driver, under **Connect to your application**, click **Drivers**.

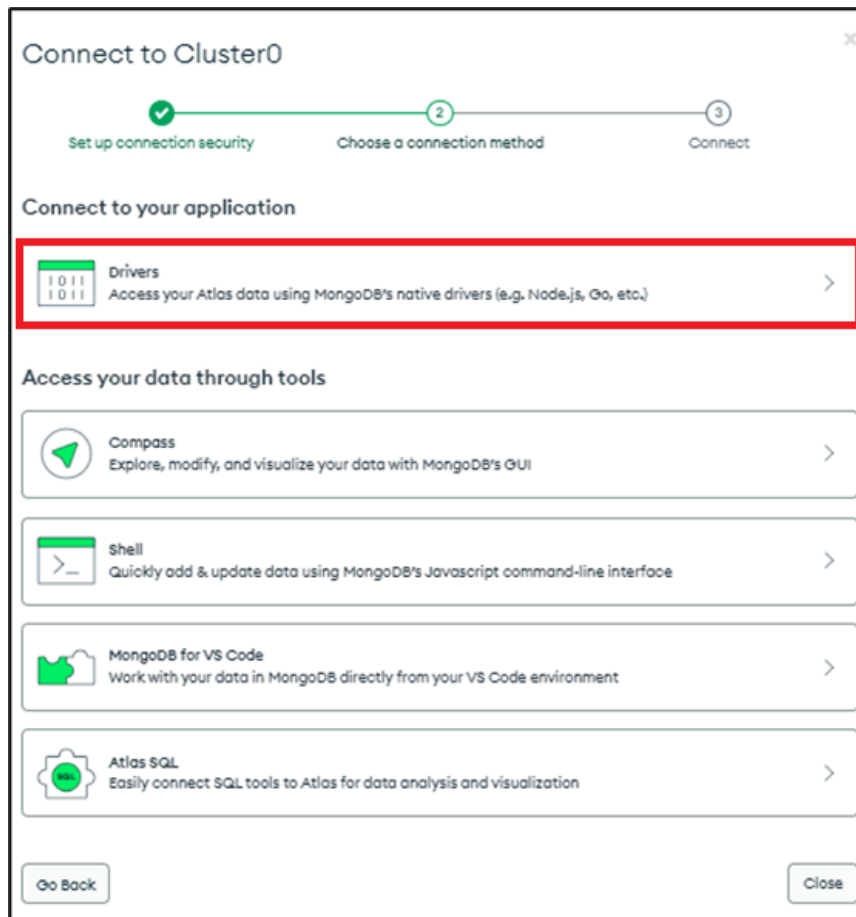


Figure 12: Choose a connection method Page

3. The **Connect** page opens as shown in Figure 13.

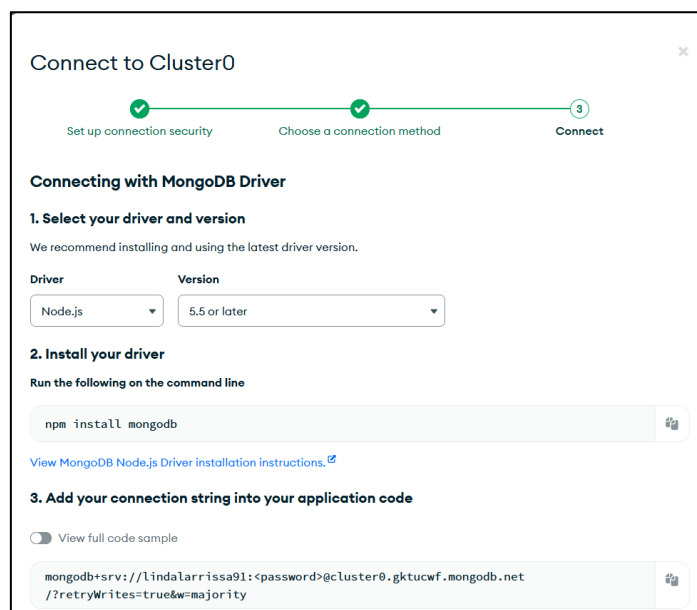


Figure 13: Connect Page

4. To select the driver and version, under **Select your driver and version**, click the **Driver** drop-down list, and select **Node.js** as shown in Figure 14.

Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver **Version**

Node.js 5.5 or later

C# / .NET
Go
Java
Kotlin
✓ Node.js
PHP
Python
Ruby
Rust

command line
db
Driver installation instructions.
Add your connection string into your application code

mongodb://lindalarrissa91:<password>@cluster0.gktucwf.mongodb.net/?
retryWrites=true&w=majority

Replace <password> with the password for the lindalarrissa91 user. Ensure any option params are URL encoded

Figure 14: Select your driver and version Page

5. The connection string is displayed. It should be copied, edited, and saved to be used in the application.
- To copy the connection string:
- Under **Add your connection string into your application code**, select the connection string as shown in Figure 15.
 - Press **Ctrl + c**. Paste the connection string in a Notepad file and save it for further use.
 - To close the **Connect to Cluster0** dialog box, click **Close**.

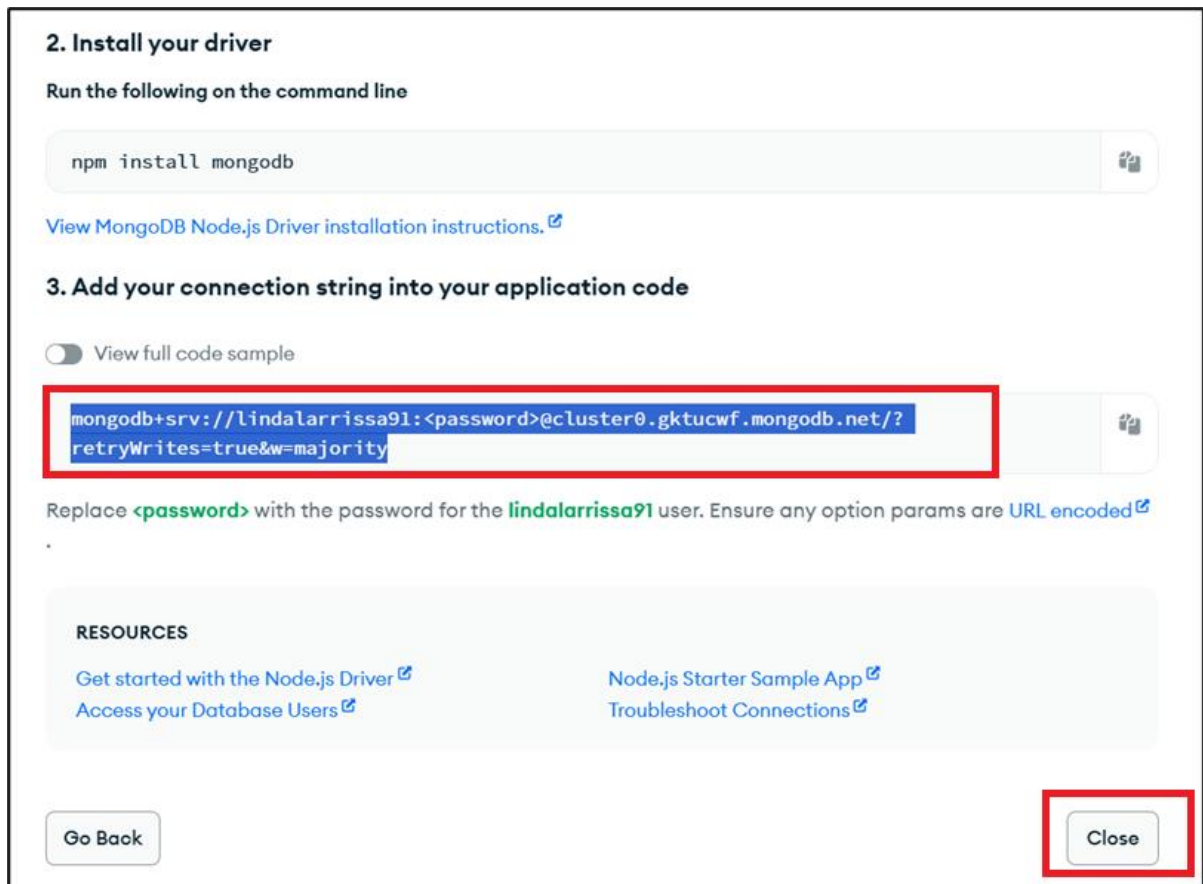


Figure 15: Copy the Connection String

Edit the Connection String

The copied connection string cannot be directly used in the application. Edit the connection string to include the password and database name. In the connection string:

- Replace `<password>` with `linda91`.
- Type the database name, `Library`, before the question mark.

The edited connection string is as follows:

`mongodb+srv://lindalarrissa91:linda91@cluster0.gktucwf.mongodb.net/Library?retryWrites=true&w=majority`

Use the edited connection string in the library application.

Appendix B

UPLOAD LIBRARY APPLICATION TO GITHUB REPOSITORY

Upload the library application to GitHub repository by performing the given steps:

1. Ensure to install the latest version of GitHub on the local system. To do so, navigate to <https://git-scm.com/downloads>, and complete the installation.

To open GitHub on the browser, navigate to <https://github.com/>.

The **Let's build from here** page appears as shown in Figure 1.

To create a new account, on the upper-right of **Let's build from here** page, click **Sign up**.

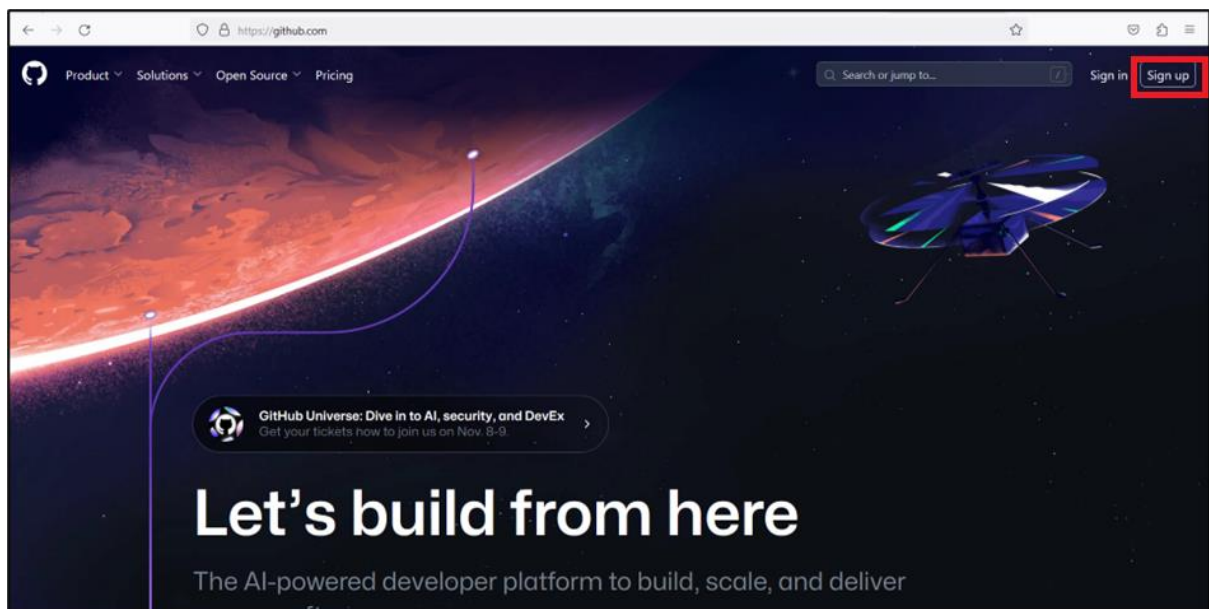


Figure 1: Let's build from here Page

2. Complete the account creation process by filling in the necessary details.

The new GitHub account is created and the dashboard is displayed as shown in Figure 2. To create a new repository, on the left of GitHub dashboard, click **Create repository**.

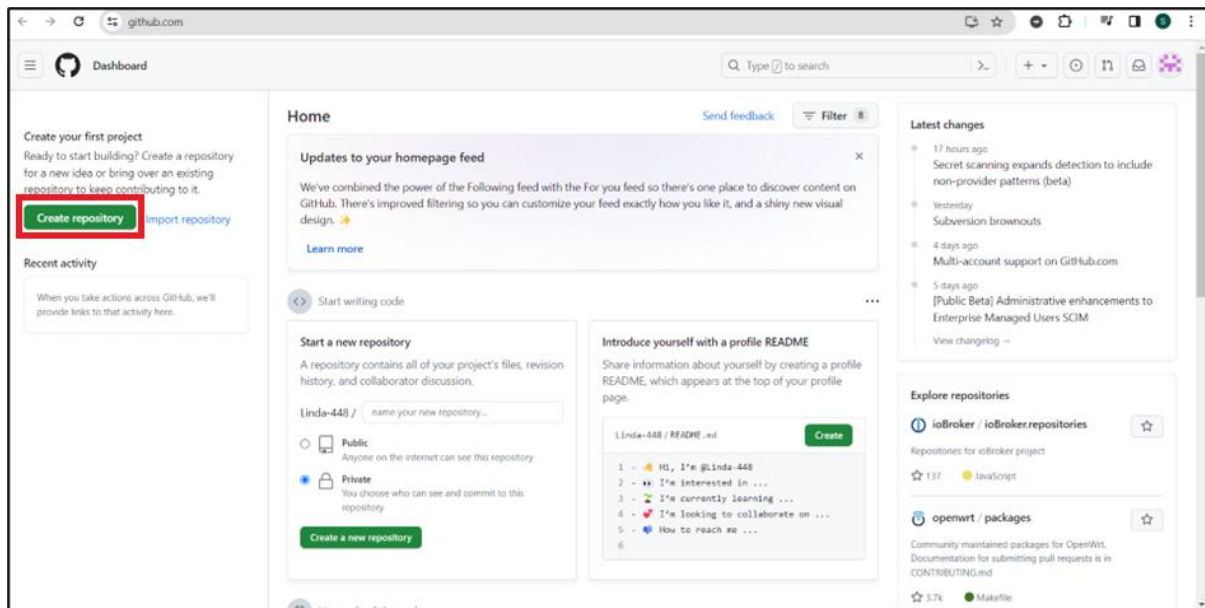


Figure 2: GitHub Dashboard

3. To provide the repository name, on the **Create a new repository** page, in the **Repository name** box, type **Library application** as shown in Figure 3.

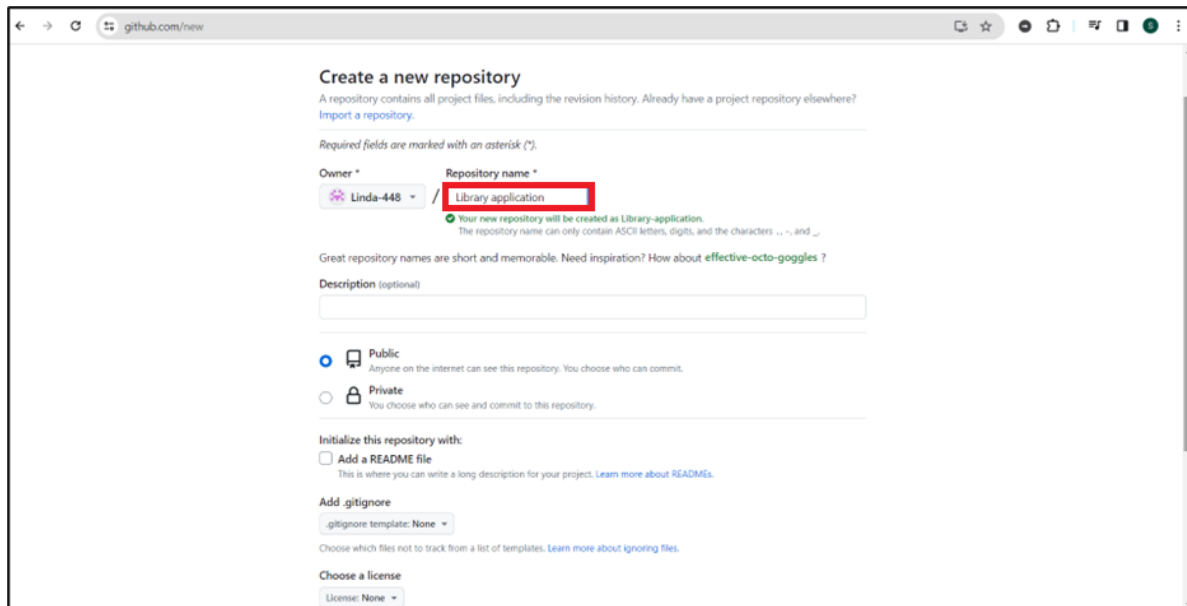


Figure 3: Create a new repository Page

4. To finish the process of creating a new repository, on the **Create a new repository** page, scroll down, and click **Create repository** as shown in Figure 4.

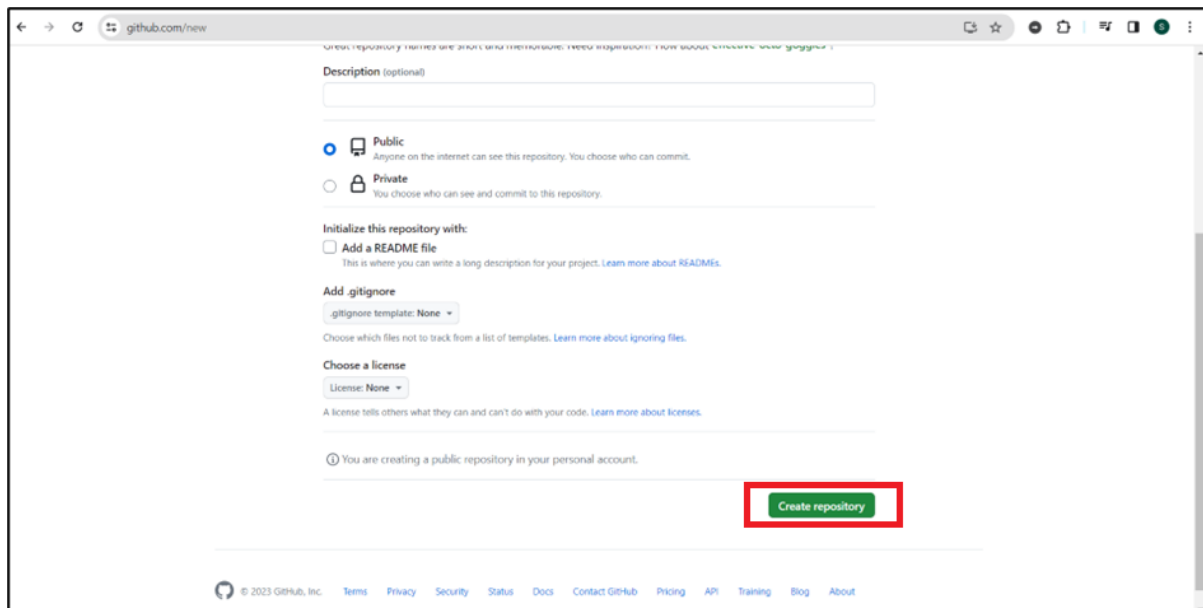


Figure 4: Creating a New Repository

5. The new repository, **Library-application**, is created as shown in Figure 5.

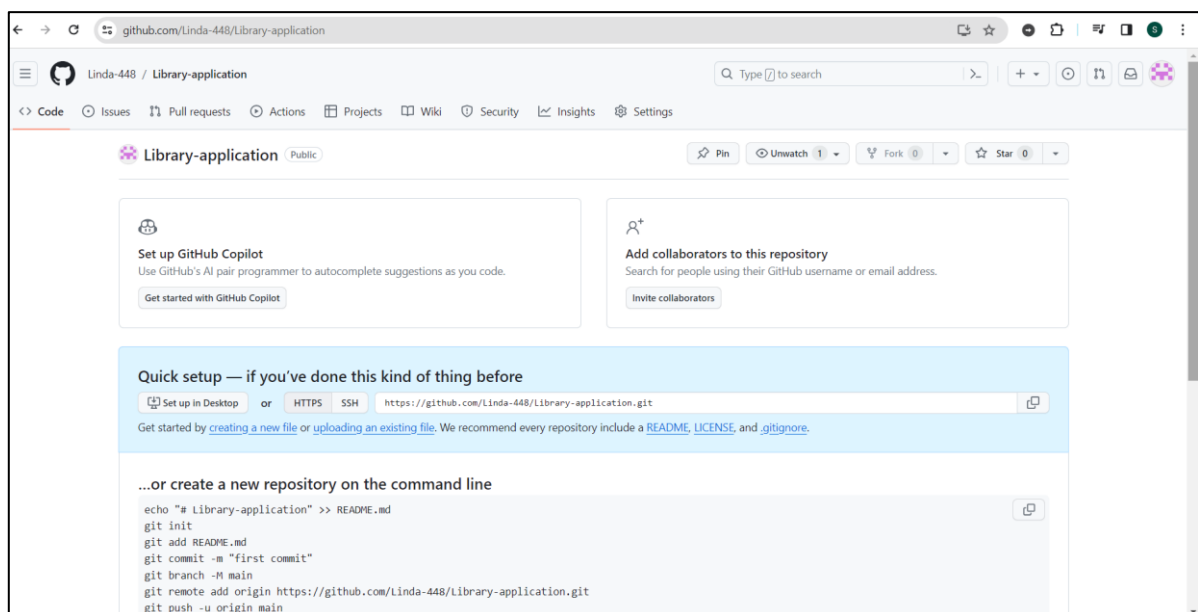


Figure 5: Library-application Repository

6. To create the new repository using the command prompt, open the Node.js prompt, and navigate to the `library_application` folder.

7. To initialize an empty Git repository, at the Command Prompt, type the command, and press **Enter** as shown in Figure 6.

```
git init
```

```
C:\NodeJS\library_application>git init  
Initialized empty Git repository in C:/NodeJS/library_application/.git/
```

Figure 6: Initializing the Application

8. To add all the application resources in the **library_application** folder, type the commands one by one by pressing **Enter** as shown in Figures 7, 8, and 9.

```
git add model  
  
git add views  
  
git add public  
  
git add app.js
```

```
C:\NodeJS\library_application>git add model  
  
C:\NodeJS\library_application>git add views  
  
C:\NodeJS\library_application>git add public  
  
C:\NodeJS\library_application>git add app.js
```

Figure 7: Adding the Folders in the Root Directory

```
git add package.json  
  
git add package-lock.json
```

```
C:\NodeJS\library_application>git add package.json
warning: LF will be replaced by CRLF in package.json.
The file will have its original line endings in your working directory

C:\NodeJS\library_application>git add package-lock.json
warning: LF will be replaced by CRLF in package-lock.json.
The file will have its original line endings in your working directory
```

Figure 8: Adding the Files in the Root Directory

```
git add node_modules
```

```
C:\NodeJS\library_application>git add node_modules
warning: LF will be replaced by CRLF in node_modules/.bin/ejs.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.bin/ejs.ps1.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.bin/jake.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.bin/jake.ps1.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.bin/mime.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.bin/mime.ps1.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/.package-lock.json.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/@mongodb-js/saslprep/LICENSE.
The file will have its original line endings in your working directory
warning: LF will be replaced by CRLF in node_modules/@mongodb-js/saslprep/dist/.esm-wrapper.mjs.
The file will have its original line endings in your working directory
```

Figure 9: Adding the Packages

9. To save the updates done to the repository, type the command and press **Enter** as shown in Figure 10.

```
git commit -m "first commit"
```

```
C:\NodeJS\library_application>git commit -m "first commit"
[master (root-commit) 46f60a0] first commit
1895 files changed, 384987 insertions(+)
create mode 100644 app.js
create mode 100644 model/Book.js
create mode 100644 model/User.js
create mode 100644 node_modules/.bin/ejs
create mode 100644 node_modules/.bin/ejs.cmd
create mode 100644 node_modules/.bin/ejs.ps1
create mode 100644 node_modules/.bin/jake
create mode 100644 node_modules/.bin/jake.cmd
create mode 100644 node_modules/.bin/jake.ps1
create mode 100644 node_modules/.bin/mime
create mode 100644 node_modules/.bin/mime.cmd
create mode 100644 node_modules/.bin/mime.ps1
create mode 100644 node_modules/.package-lock.json
create mode 100644 node_modules/@mongodb-js/saslprep/LICENSE
create mode 100644 node_modules/@mongodb-js/saslprep/dist/.esm-wrapper.mjs
create mode 100644 node_modules/@mongodb-js/saslprep/dist/code-points-data.d.ts
create mode 100644 node_modules/@mongodb-js/saslprep/dist/code-points-data.d.ts.map
create mode 100644 node_modules/@mongodb-js/saslprep/dist/code-points-data.js
create mode 100644 node_modules/@mongodb-js/saslprep/dist/code-points-data.js.map
create mode 100644 node_modules/@mongodb-js/saslprep/dist/code-points-src.d.ts
```

Figure 10: Committing the Changes to GitHub Repository

10. To rename the old branch name to `main`, type the command, and press **Enter** as shown in Figure 11.

```
git branch -M main
```

```
C:\NodeJS\library_application>git branch -M main
```

Figure 11: Renaming the Branch

11. To add a remote repository for the existing repository at the given URL, type the command and press **Enter** as shown in Figure 12.

```
git remote add origin https://github.com/Linda-448/Library-application.git
```

```
C:\NodeJS\library_application>git remote add origin https://github.com/Linda-448/Library-application.git
```

Figure 12: Adding a Remote Repository

12. To add the upstream reference for the branch that is successfully pushed, type the command and press **Enter** as shown in Figure 13.

```
git push -u origin main
```

```
C:\NodeJS\library_application>git push -u origin main
```

Figure 13: Adding Upstream Reference

13. The **Connect to GitHub** page appears. To connect to GitHub, on the **Connect to GitHub** page, click **Sign in with your browser** as shown in Figure 14.

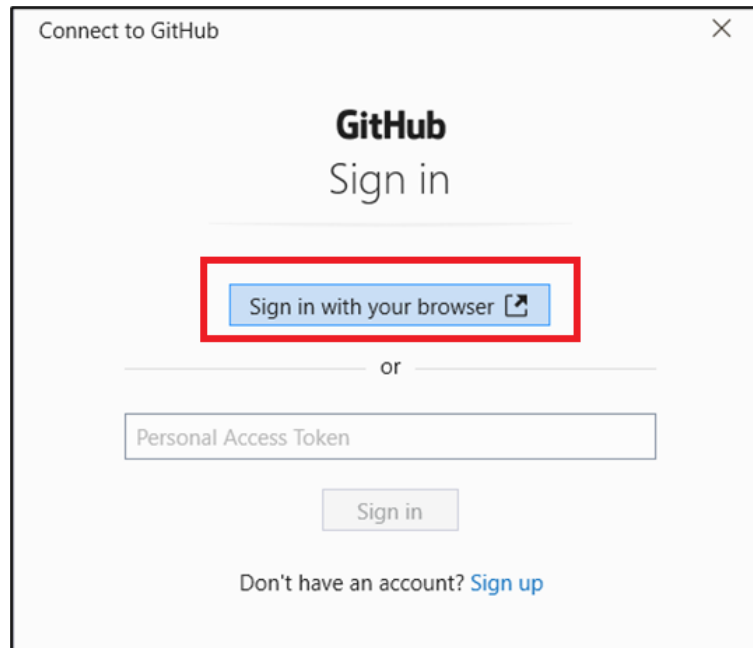


Figure 14: Connect to GitHub Page

14. To authorize the Git Ecosystem, on the **Authorize Git Credential Manager** page, click **Authorize git-ecosystem** as shown in Figure 15.

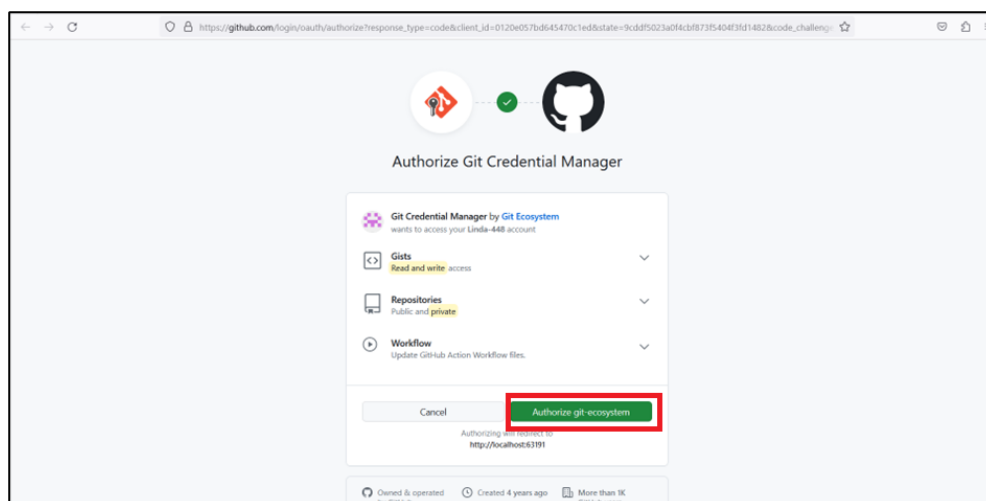


Figure 15: Authorize Git Credential Manager Page

15. On the **Authentication Succeeded** page, the message “**Authentication Succeeded**” is displayed as shown in Figure 16.

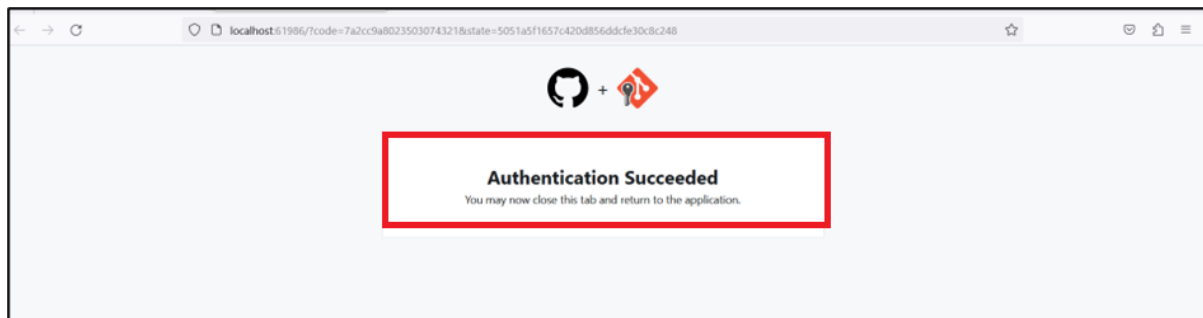


Figure 16: Authentication Succeeded Page

On some computers, the Git Credential Manager may not work properly or may be blocked by a firewall. In such cases, the developer can use following command and substitute the password parameter with the value of a personal access token:

```
git remote set-url origin
https://username:password@github.com/organization/repo.git
```

Refer to the following link to know how to create a Personal Access Token:

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>

16. To access the newly created repository, navigate to the GitHub dashboard, and click **Library-application** as shown in Figure 17.

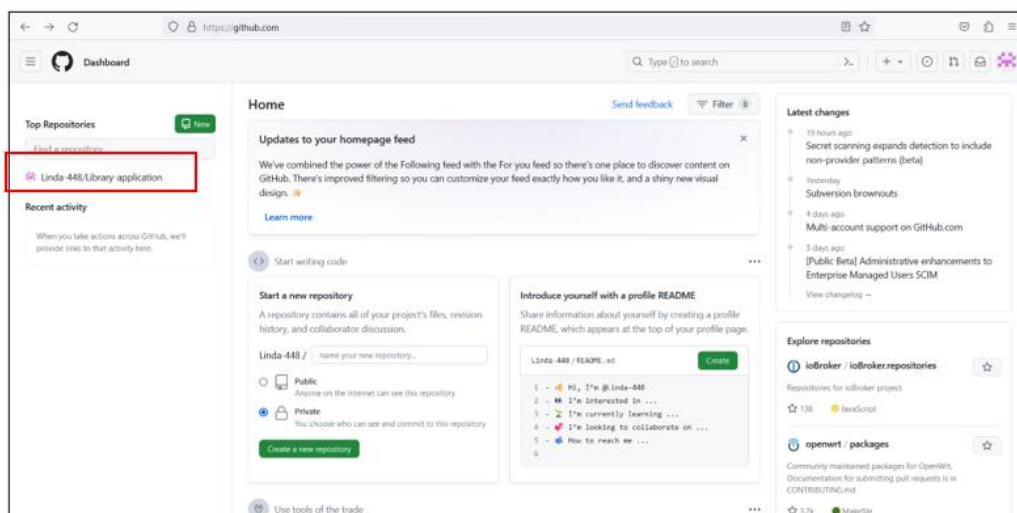


Figure 17: GitHub Dashboard Page

17. The **Library-application** repository is created and all the resources are

uploaded as shown in Figure 18.

The library application is successfully uploaded to the GitHub repository.

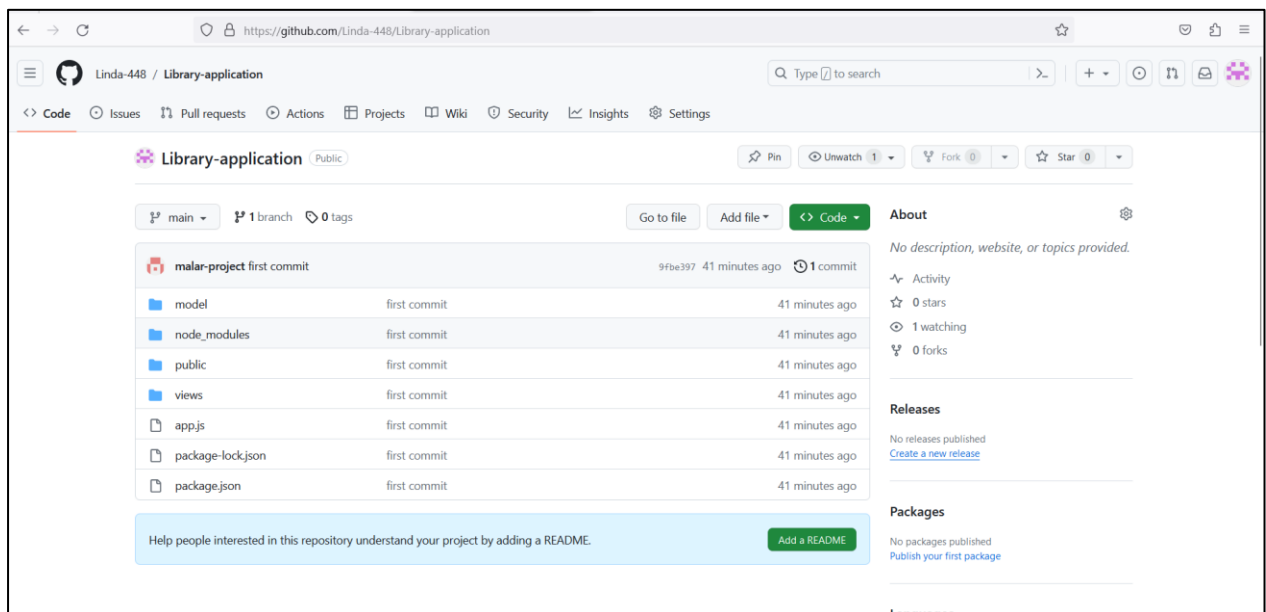


Figure 18: Library Application Resources on the GitHub

Appendix C

Sr. No.	Case Studies																												
1.	Create a simple hospital management application using Node.js. Table 1 lists some sample patient data. <table><tr><th>Patient_Name</th><th>Age</th><th>Diagnosis</th><th>Treatment_Cost (\$)</th></tr><tr><td>John</td><td>34</td><td>Malaria</td><td>1177</td></tr><tr><td>Emily</td><td>29</td><td>Influenza</td><td>2167</td></tr><tr><td>Lara</td><td>23</td><td>Mpox</td><td>3423</td></tr><tr><td>Emma</td><td>50</td><td>Typhoid Fever</td><td>1233</td></tr><tr><td>Jack</td><td>45</td><td>Malaria</td><td>1177</td></tr><tr><td>Mike</td><td>40</td><td>Zika Virus</td><td>4562</td></tr></table> Table 1: Patient Details a. Create a function for managing patients using Patient_Name, Age, Diagnosis, and Treatment_Cost parameters. b. Save the treatment costs of five patients from Table 1 in an array. Then, use a for loop to calculate the total treatment cost and average treatment cost. c. Create an empty patientRecordFile.txt file. Write the first three patient details from Table 1 to this file. d. Read and display contents of patientRecordFile.txt. e. Append details of Mike to patientRecordFile.txt. f. Delete patientRecordFile.txt file.	Patient_Name	Age	Diagnosis	Treatment_Cost (\$)	John	34	Malaria	1177	Emily	29	Influenza	2167	Lara	23	Mpox	3423	Emma	50	Typhoid Fever	1233	Jack	45	Malaria	1177	Mike	40	Zika Virus	4562
Patient_Name	Age	Diagnosis	Treatment_Cost (\$)																										
John	34	Malaria	1177																										
Emily	29	Influenza	2167																										
Lara	23	Mpox	3423																										
Emma	50	Typhoid Fever	1233																										
Jack	45	Malaria	1177																										
Mike	40	Zika Virus	4562																										
2.	a. Create a simple HTTP server that responds with a plain text message,Welcome to the Evergreen Health Care Hospital. b. Create an event-driven login notification system by registering an event listener for the events listed in Table 2. <table><tr><th>Events</th><th>Time (in secs)</th></tr><tr><td>Patient Admitted</td><td>100</td></tr><tr><td>Preliminary Checkup Completed</td><td>150</td></tr><tr><td>Doctor Visit</td><td>800</td></tr></table> Table 2: Event Details c. Parse the query string using this data: Patient_Name: Jack Age: 23 Diagnosis: Food-borne and water-borne	Events	Time (in secs)	Patient Admitted	100	Preliminary Checkup Completed	150	Doctor Visit	800																				
Events	Time (in secs)																												
Patient Admitted	100																												
Preliminary Checkup Completed	150																												
Doctor Visit	800																												

	Extract data from the URL query string and display parsed data in the console. d. Convert JavaScript objects into URL query strings. - Define two patient objects, <code>patient1</code> and <code>patient2</code>, each with distinct properties such as <code>Patient_Name</code>, <code>Activity_Status</code>, and <code>Patient_Condition</code> using the given data: <pre> patient1 Patient_Name = Jack Activity_Status = false Patient_Condition = stable patient2 Patient_Name = Daniel Activity_Status = false Patient_Condition = Critical </pre> - To serialize the object, use <code>patient2</code>. - Use the <code>stringify</code> method on the object. - Display the result on the console. e. Encode the given patient data: <pre> Patient_Name = Jack Age = 40 Patient_Condition = stable </pre> f. Decode the URL <pre> patientName=Lara&patientGender=Female&AverageGlucose Level=229&available=true </pre> g. Send an email to the Medical Insurance company with the subject Medical Claim and add body text as Hi I am Jack and I want to submit a medical claim. h. Create a simple Web application using Express.js as the backend to display patient information. Refer to Table 1 for patient details.
3.	a. Create a text file using Notepad with the given statements: <pre> Statement 1: Jack's condition is not stable yet. Statement 2: Now, Jack is feeling a little better. </pre> Perform read operation on the text file using the <code>async</code> and <code>await</code> functions.

	• Display <code>Statement 1</code> after three seconds and <code>Statement 2</code> after four seconds. b. Create a simple RESTful API for patient details. This RESTful API manages patient details using Node.js and Express using Table 1. • Create a new patient. • Update the patient's name <code>John</code> to <code>Kevin</code>. • Delete a patient whose age is 50. • Display the results on the console. • Filter the patient data based on a specified diagnosis such as <code>Malaria</code>. • Add a new patient using a custom method and non-standard URL. • Update the patient's name <code>Mike</code> to <code>Mark</code> using a custom method and non-standard URL. • Delete a patient whose age is greater than 44 using a custom method and non-standard URL. • Display the results on the console. c. Perform the CRUD operations with the data given in Table 1. • Create a MongoDB database <code>Patient</code> and collection <code>Patient_Collection</code> and insert the first three documents given in Table 1 into the <code>Patient_Collection</code>. • View the inserted documents. • Update Name: <code>Lara</code> as <code>Anna</code> and age of <code>Emily</code> as 56. • Delete a document where Name is equal to <code>Emily</code>. • Display the results.
--	---